BANK ĊENTRALI TA' MALTA
EUROSISTEMA
CENTRAL BANK OF MALTA

November 17, 2025

Deep Learning for Time Series Forecasting with RNNs and LSTMs

Facilitator: Mr Bilal Kurban, Ms Priyanka Bairapura, Ms Roxanne Spiteri (CBM)

1 Introduction

Time series forecasting is defined as the process of **predicting future values based on historical observations ordered in time**. Crucially, unlike traditional regression problems, time series data possesses **temporal dependencies**, meaning the sequence and order of observations matter, and past values inherently influence future outcomes. In this project, we are going to try to forecast Malta's Foreign Trade in the Central Bank of Malta Website.

1.1 Reason for Using Deep Learning

While time series forecasting is a critical component of business intelligence and decision-making, traditional statistical methods—such as **ARIMA** (which requires stationarity and manual parameter selection) or **Exponential Smoothing** (limited for complex relationships)—demand extensive feature engineering and domain knowledge.

Deep learning approaches, particularly **Recurrent Neural Networks (RNNs)** and **Long Short-Term Memory networks (LSTMs)**, overcome these limitations by offering significant advantages:

- **Automatic feature learning:** They eliminate the need for manual feature engineering.
- **Complex patterns:** They are capable of capturing non-linear relationships.
- **Multiple variables:** They can easily handle multivariate forecasting scenarios.
- **Scalability:** They are efficient when working with large datasets.

1.2 Business Applications

The techniques taught are applicable across various critical business functions, including:

- **Trade forecasting** (e.g., predicting imports or exports for resource planning).
- **Demand forecasting** (essential for inventory management and supply chain optimization).
- **Financial prediction** (forecasting stock prices or revenue).
- **Resource planning** (such as determining energy consumption or staffing requirements).

2 Data Loading and Exploration

2.1 Imported Libraries

The goal here is simply to **arm ourselves with the right tools**. We need tools for data manipulation (`pandas`, `numpy`), tools for visual diagnosis (`matplotlib`, `seaborn`), tools for preparing the data (`sklearn.preprocessing`), and, of course, the core deep learning framework (`tensorflow` and `keras`).

Data Manipulation: - `pandas`: For handling tabular data and time series operations - `numpy`: For numerical computations and array operations

Visualization: - `matplotlib.pyplot`: For creating static plots - `seaborn`: For statistical visualizations with better aesthetics

Preprocessing: - `sklearn.preprocessing.MinMaxScaler`: To normalize data (essential for neural networks) - `sklearn.metrics`: For evaluation metrics (RMSE, MAE)

Deep Learning: - `tensorflow`: The backend framework - `keras`: High-level API for building neural networks - `keras.layers`: LSTM, Dense, Dropout layers - `keras.optimizers`: Adam optimizer for training

1. **`pandas` and `numpy`:** These are the bedrock for handling data in Python. We use `pandas` specifically for working with tabular data and performing essential time series operations, while `numpy` handles the heavy lifting of array operations needed later when we format the data for Keras.
2. **`MinMaxScaler`:** We explicitly choose this scaler because deep neural networks, especially RNNs and LSTMs, are very sensitive to input scale. Scaling our data, usually to a range of 0 to 1, prevents training instability and helps convergence. `MinMaxScaler` works particularly well with Sigmoid or Tanh activation functions, which are common in recurrent layers.
3. **`keras`:** We choose Keras because it's a high-level API built on TensorFlow. This makes defining complex architectures (like stacking LSTMs and Dropouts) much faster and simpler than working directly with the backend framework.

2.1.1 Other Alternatives

- **Different Scalers:** If your data contains significant outliers, you might opt for the `StandardScaler` (mean=0, std=1), which is less sensitive to extreme values. Alternatively, the `RobustScaler` is even better for handling outliers since it uses the median and Interquartile Range.
- **Visualization:** While `matplotlib` and `seaborn` are standard, you could use interactive libraries like Plotly for dynamic visualizations.

2.1.2 Tips and Tricks

- **Reproducibility:** A critical step, even if it seems small, is to always **set random seeds** (for NumPy and TensorFlow/Keras). This ensures that if you run the notebook again, the results will be identical.

[4]:
```
# Code Block 1: Library Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout, SimpleRNN
from tensorflow.keras.optimizers import Adam
import warnings
warnings.filterwarnings('ignore')

# Set random seeds for reproducibility
np.random.seed(41)
tf.random.set_seed(41)

# Load the trade data
url='https://raw.githubusercontent.com/CBM-Statistics/CBM-Statistics/refs/heads/
  ↪(root)/Trade.csv'
df = pd.read_csv(url)
print("Dataset shape:", df.shape)
print("\nColumn information:")
print(df.info())
print("\nFirst few rows:")
print(df.head())
```

```
Dataset shape: (272, 5)

Column information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 272 entries, 0 to 271
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Year              272 non-null    int64
 1   Month             272 non-null    int64
 2   Exports           272 non-null    float64
 3   Imports           272 non-null    float64
 4   Balance of trade  272 non-null    float64
dtypes: float64(3), int64(2)
memory usage: 10.8 KB
None

First few rows:
   Year  Month  Exports  Imports  Balance of trade
```

```
0  2003      1    169.3    223.9            -54.5
1  2003      2    160.0    241.8            -81.8
2  2003      3    185.4    250.9            -65.5
3  2003      4    180.1    272.5            -92.5
4  2003      5    174.9    251.1            -76.2
```

2.2 Understanding the Output

What to look for:

- **Shape**: Number of rows (time points) and columns (variables)
- **Data types**: Should be numeric for forecasting
- **Column names**: Understand what each variable represents
- **Sample values**: Check if data looks reasonable

2.3 Data Loading, Inspection and DateTime Indexing

Here we load the CSV/file and set the temporal index)

First, we load the data. Second and crucially, we ensure the time column is correctly understood by pandas as time points, turning it into the **DateTime Index**. We also run quick checks (`.head()`, `.shape`, `.info()`) to understand the structure: how many rows (time points), how many columns (variables), and what the data types are.

Why Create a DateTime Index? This choice is essential for any serious time series analysis. A proper DateTime index provides four major benefits:

1. **Easy Slicing:** You can easily filter the data by date (e.g., '2019-01-01' onwards).
2. **Date Arithmetic:** Pandas can automatically handle things like monthly differences or calculating time lags.
3. **Better Visualization:** The time axis will be displayed correctly and uniformly.
4. **Resampling:** It enables powerful operations like aggregating daily data to monthly data, or vice versa.

2.3.1 Other Alternatives

You could process the data *without* a DateTime index, but you would have to manually handle all time-based calculations, which introduces complexity and potential error.

2.3.2 Tips and Tricks

- **Chronological Order:** Always **ensure your time series is sorted chronologically**. If it's not, the sequence creation steps (Code Block 5) will be broken.
- **Look for Errors:** Immediately check the output shape and data types. Time series variables should ideally be numeric.

[7]:
```python
# Code Block 2 - Data exploration
print("\nDescriptive statistics:")
print(df.describe())

# Check for missing values
```

```python
print("\nMissing values:")
print(df.isnull().sum())

# Create a proper datetime index
# This is CRUCIAL for time series analysis
df['Date'] = pd.to_datetime(df[['Year', 'Month']].assign(day=1))
df.set_index('Date', inplace=True)
df.sort_index(inplace=True)

print(f"\nTime series spans from {df.index.min()} to {df.index.max()}")
print(f"Total months: {len(df)}")
```

```
Descriptive statistics:
              Year       Month     Exports      Imports  Balance of trade
count   272.000000  272.000000  272.000000   272.000000        272.000000
mean   2013.838235    6.441176  286.642279   484.192647       -197.553309
std       6.557405    3.446512   92.841682   200.515362        145.546863
min    2003.000000    1.000000  138.800000   215.300000       -935.700000
25%    2008.000000    3.000000  206.975000   321.075000       -275.675000
50%    2014.000000    6.000000  281.950000   456.050000       -155.850000
75%    2019.250000    9.000000  356.250000   605.550000        -94.950000
max    2025.000000   12.000000  669.400000  1312.100000         -3.300000

Missing values:
Year                0
Month               0
Exports             0
Imports             0
Balance of trade    0
dtype: int64

Time series spans from 2003-01-01 00:00:00 to 2025-08-01 00:00:00
Total months: 272
```

2.4 Visualizing the Data

We are creating line plots, usually of individual variables over time. The purpose is **visual diagnosis**—we need to see what we are trying to predict.

We use the time axis on the X-axis because this is the only way to accurately identify three key patterns:

1. **Trends:** Is the data generally increasing, decreasing, or stationary over the long term?
2. **Seasonality:** Are there repeating patterns (e.g., repeating every 12 months for yearly seasonality)?
3. **Outliers/Anomalies:** Are there sudden, unexpected spikes or drops?

2.4.1 Other Alternatives

While a simple line plot is essential, for statistical time series analysis, you might also generate **autocorrelation plots** to numerically identify lagged dependencies.

2.4.2 Tips and Tricks

- **Self-Correction:** Before moving on, use these plots to answer key questions: Do I need a long or short sequence length? (If I see annual seasonality, I probably need at least 12 steps). Does the data look clean, or do I need robust scaling due to outliers?

[9]:
```
# Code Block 3 - Visualize the time series
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Plot exports
axes[0, 0].plot(df.index, df['Exports'], color='blue', alpha=0.7)
axes[0, 0].set_title('Monthly Exports Over Time', fontsize=12,␣
 ↪fontweight='bold')
axes[0, 0].set_ylabel('Exports (in units)')
axes[0, 0].grid(True, alpha=0.3)

# Plot imports
axes[0, 1].plot(df.index, df['Imports'], color='red', alpha=0.7)
axes[0, 1].set_title('Monthly Imports Over Time', fontsize=12,␣
 ↪fontweight='bold')
axes[0, 1].set_ylabel('Imports (in units)')
axes[0, 1].grid(True, alpha=0.3)

# Plot balance of trade
axes[1, 0].plot(df.index, df['Balance of trade'], color='green', alpha=0.7)
axes[1, 0].set_title('Balance of Trade Over Time', fontsize=12,␣
 ↪fontweight='bold')
axes[1, 0].set_ylabel('Balance of Trade')
axes[1, 0].grid(True, alpha=0.3)

# Plot all series together
axes[1, 1].plot(df.index, df['Exports'], label='Exports', alpha=0.7)
axes[1, 1].plot(df.index, df['Imports'], label='Imports', alpha=0.7)
axes[1, 1].plot(df.index, df['Balance of trade'], label='Balance of Trade',␣
 ↪alpha=0.7)
axes[1, 1].set_title('All Series Combined', fontsize=12, fontweight='bold')
axes[1, 1].set_ylabel('Value')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Participant Exercise: After viewing the plots:

1. Is there a trend in the data? (Increasing/Decreasing/Stationary)
2. Do you see any seasonal patterns?
3. Are there any sudden spikes or drops?
4. Which variable has the most volatility?

3 Data Preprocessing

3.1 Scaling the Data

Here we fit and transform the data using MinMaxScaler. The below code block implements the scaling discussed earlier. It uses the chosen scaler (`MinMaxScaler` in our case) to normalize the numeric values of the data.

This is where students often make the most dangerous mistake: **data leakage**.

1. We must first perform the chronological **Train-Test Split** (Code Block 6).
2. We then **fit the scaler *only* on the training data**. This means the scaler learns the min/max values based purely on historical observations.
3. We use that *fitted* scaler to transform both the training data and the testing data. This ensures the testing data remains unseen and is scaled based on the statistics of the past, simulating a real-world scenario where you wouldn't know future statistics.

3.1.1 Tips and Tricks

- **Gradient Health:** If you skip scaling, training will be unstable, convergence will take longer, or gradients can explode or vanish—all leading to degraded model performance. Scaling is not optional for deep learning!

[12]:
```
# Code Block 4 - Data scaling
def create_sequences(data, seq_length, target_col_idx=0):
    """
    Create sequences for time series prediction

    Args:
        data: numpy array of time series data
        seq_length: length of input sequences (lookback period)
        target_col_idx: index of target column for prediction

    Returns:
        X: input sequences of shape (samples, seq_length, features)
        y: target values of shape (samples,)

    Example:
        If seq_length=3 and data=[1,2,3,4,5]:
        X = [[1,2,3], [2,3,4]], y = [4, 5]
    """
    X, y = [], []
    for i in range(len(data) - seq_length):
        # Take seq_length consecutive values as input
        X.append(data[i:(i + seq_length)])
        # The next value after the sequence is the target
        y.append(data[i + seq_length, target_col_idx])
    return np.array(X), np.array(y)

# For this demonstration, we'll focus on forecasting Exports
# You can easily modify this to forecast Imports or Balance of Trade
target_variable = 'Exports'
print(f"Target variable: {target_variable}")

# Prepare data for modeling
# Reshape to 2D array (required by sklearn's scaler)
data = df[target_variable].values.reshape(-1, 1)

# Normalize the data
# WHY? Neural networks work best with values between 0 and 1
# MinMaxScaler: X_scaled = (X - X_min) / (X_max - X_min)
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)

print(f"Original data shape: {data.shape}")
```

```
print(f"Original data range: [{data.min():.2f}, {data.max():.2f}]")
print(f"Scaled data range: [{scaled_data.min():.3f}, {scaled_data.max():.3f}]")
```

```
Target variable: Exports
Original data shape: (272, 1)
Original data range: [138.80, 669.40]
Scaled data range: [0.000, 1.000]
```

3.1.2 Why MinMaxScaler vs Other Scalers?

MinMaxScaler (0 to 1):

- Works well with sigmoid/tanh activations
- Preserves zero values
- Good for bounded data
- Sensitive to outliers

StandardScaler (mean=0, std=1): - Better for data with outliers - Works well with ReLU activations - Can produce values outside [0,1]

RobustScaler: - Very robust to outliers (uses median, IQR) - More complex interpretation

Recommendation: Start with MinMaxScaler, try others if you have outliers

3.2 Creating Sequences

Here we define and run the function that turns the scaled data into 3D arrays.

This is arguably the most crucial preprocessing step, defining **"The Heart of Time Series DL"**. We transform the flat, continuous time series array into a supervised learning format using a **sliding window** approach. This converts the data from:

> *One row (time step) = one sample* to: *A window of past values = one sample*

The function outputs the required 3D input shape for RNNs/LSTMs: `(samples, timesteps, features)`.

The Concept:

Traditional ML: Each row is independent → `[feature1, feature2, ...] → target`

Time Series DL: We use a **window** of past values → `[t-n, t-n+1, ..., t-1] → t`

Example:

If sequence length = 3, and data = [10, 20, 30, 40, 50]:

- Input 1: [10, 20, 30] → Target: 40
- Input 2: [20, 30, 40] → Target: 50

This is called a **sliding window** approach.

3.2.1 Sequence Length Choice

The choice of the `seq_length` (or `lookback window`) is key.

- If it's **too short** (e.g., 3-6), the model might miss important long-term relationships, like annual seasonality.
- If it's **too long** (e.g., 24+), it requires significantly more data and can make the model harder to train or prone to overfitting.
- For many monthly datasets, a **sweet spot like 12** is chosen because it ensures the model can always see a full year of history.

3.2.2 Other Alternatives

- **Multi-step output:** Instead of the target being just the next value (t), the output could be the next k values simultaneously.
- **Encoder-Decoder Architectures:** These are better suited for scenarios requiring very long forecast horizons.
- **Attention mechanisms**: For very long sequences

3.2.3 Tips and Tricks

- **Data Loss:** Remember that when you create sequences, you **lose a number of samples equal to your `seq_length`** at the beginning of the dataset. If you have 268 total points and a sequence length of 12, you will only have 256 training samples.
- **Shape is King:** Always verify the shape of your output `X` array is **3D**: `(samples, timesteps, features)`. If it's not 3D, the Keras recurrent layer won't accept it.

3.2.4 Parameters Explained:

- `data`: Your time series array
- `seq_length`: How many past time steps to use (lookback window)
- `target_col_idx`: Which column to predict (0 for univariate)

[15]:
```
# Code Block 5 - Creating Sequences
# Define sequence length (lookback window)
# CRITICAL DECISION: This affects model performance significantly!
SEQUENCE_LENGTH = 12  # Use 12 months of history to predict next month

print(f"Sequence length: {SEQUENCE_LENGTH} months")
print(f"Rationale: 12 months captures yearly seasonality")

# Create sequences
X, y = create_sequences(scaled_data, SEQUENCE_LENGTH)
print(f"\nInput sequences shape: {X.shape}")
print(f"  - {X.shape[0]} samples")
print(f"  - {X.shape[1]} time steps per sample")
print(f"  - {X.shape[2]} feature(s) per time step")
print(f"Target values shape: {y.shape}")
```

```
Sequence length: 12 months
Rationale: 12 months captures yearly seasonality

Input sequences shape: (260, 12, 1)
```

```
  - 260 samples
  - 12 time steps per sample
  - 1 feature(s) per time step
Target values shape: (260,)
```

3.2.5 Understanding the Shapes

X shape: (samples, timesteps, features)

- `samples`: Number of training examples
- `timesteps`: Length of each sequence (SEQUENCE_LENGTH)
- `features`: Number of variables (1 for univariate, >1 for multivariate)

y shape: (samples,) - One target value per sample

3.3 Train-Test Split (Implementation)

Here we slice the 3D arrays chronologically.

The below code simply implements the necessary split by dividing the prepared data into a training set (the past) and a testing set (the future).

3.3.1 The Chronological Split

This cannot be overstated: **DO NOT use `train_test_split()` without `shuffle=False`!** Randomly shuffling the data destroys the temporal dependencies, which is the entire point of time series analysis. We use a chronological split (e.g., 80% for training, 20% for testing) to ensure a realistic evaluation.

- `train_test_split()` shuffles data → destroys temporal order. When you set shuffle=False, the function does not randomize the order of the data before splitting — it simply takes the first part for training and the last part for testing.
- Use chronological split → train on past, test on future

3.3.2 Other Alternatives

- **Split Ratios:** While 80-20 is standard, you might use 70-30 or 90-10 based on how much data you have.
- **Validation Set:** For production systems, the best practice is to separate your data into three chunks: Train (60%), Validation (20%), and Test (20%). The validation set helps tune hyperparameters without touching the final, unbiased test set.

3.3.3 Tips and Tricks

- **Real-World Problem:** If you train on the data from 2010-2020 and then test on 2005-2010, the model is tested on data where the underlying patterns might be completely different. This is problematic for deployment, as patterns in time series often change (non-stationarity). **Always test on the most recent, unseen data.**

[18]:
```
# Code Block 6 - Split into train and test sets
# CRITICAL: We use temporal split, NOT random split!
```

```python
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

print(f"\nTraining set: {X_train.shape[0]} samples ({train_size/len(X)*100:.
  ↪1f}%)")
print(f"Test set: {X_test.shape[0]} samples ({(len(X)-train_size)/len(X)*100:.
  ↪1f}%)")

# Show date ranges
train_dates = df.index[SEQUENCE_LENGTH:SEQUENCE_LENGTH+train_size]
test_dates = df.index[SEQUENCE_LENGTH+train_size:]
print(f"\nTraining period: {train_dates[0]} to {train_dates[-1]}")
print(f"Test period: {test_dates[0]} to {test_dates[-1]}")
```

```
Training set: 208 samples (80.0%)
Test set: 52 samples (20.0%)

Training period: 2004-01-01 00:00:00 to 2021-04-01 00:00:00
Test period: 2021-05-01 00:00:00 to 2025-08-01 00:00:00
```

Common Mistakes to Avoid:

1. Using future data in training (data leakage)
2. Random shuffling of time series data
3. Fitting scaler on entire dataset (should fit only on training data)
4. Not checking for temporal gaps in data

4 Understanding RNNs and LSTMs

4.1 The Problem with Traditional Neural Networks

Feedforward Networks:

`Input → Hidden Layer → Output`

- Each input is processed independently
- No memory of previous inputs
- Cannot handle sequential dependencies

Example Failure: Sentence: "The clouds are in the _____"

- Feedforward NN: Doesn't know "clouds" was mentioned
- RNN: Remembers "clouds" → predicts "sky"

4.2 Recurrent Neural Networks (RNNs)

The Key Innovation: Memory

```
Hidden State (t-1)
                        ↓
Input (t) → RNN Cell → Output (t)
                        ↓
            Hidden State (t) → ...
```

How It Works:

1. Takes current input + previous hidden state
2. Produces output + new hidden state
3. New hidden state = "memory" of sequence so far

Advantages:

- Handles sequences of any length
- Parameter sharing across time steps
- Can process sequential information

Limitations:

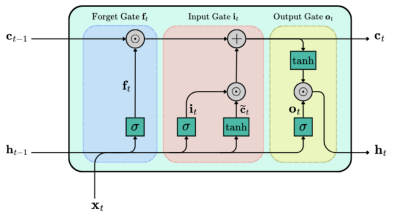
- **Vanishing gradients**: Struggles with long sequences (>10-20 steps)
- **Exploding gradients**: Unstable training
- Cannot learn long-term dependencies (e.g., can't connect info 50 steps apart)

4.3 Long Short-Term Memory (LSTM)

The Solution to RNN's Problems

LSTMs add a **cell state** (long-term memory) and **three gates**:

1. **Forget Gate**: What to remove from memory
   - Decides what information to discard from cell state
   - Values close to 0 = forget, close to 1 = keep
2. **Input Gate**: What new information to add
   - Decides which values to update
   - Creates new candidate values
3. **Output Gate**: What to output
   - Decides what parts of cell state to output



LSTM Cell, bias omitted for simplicity (adapted from Zhang et al. (2023))

Visual Analogy:

Think of LSTM as a conveyor belt:

- **Cell state**: The conveyor belt (long-term memory)
- **Forget gate**: Worker removing old boxes
- **Input gate**: Worker adding new boxes
- **Output gate**: Worker selecting which boxes to ship

Why LSTMs Work Better:

- Learns dependencies 100+ steps apart
- Mitigates vanishing gradient problem

- Selective memory (keeps relevant, forgets irrelevant)
- More stable training

Trade-offs:

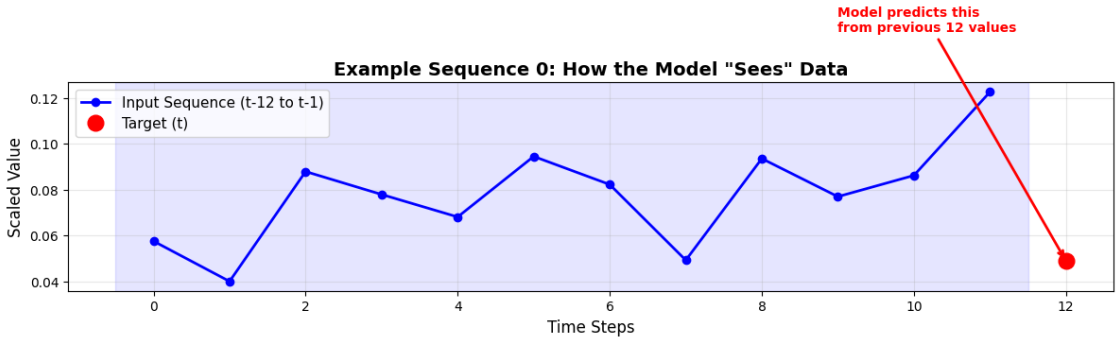
- More parameters (4x more than simple RNN)
- Slower training
- Needs more data

4.4 (Optional) Visualizing Sequence Structure

Purpose: Understand what the model actually sees during training

[21]:
```
# Code Block Optional: Visualizing Sequence Structure
# Visualize the sequence structure
def plot_sequence_example(X, y, seq_idx=0):
    """
    Plot an example sequence and its target

    Purpose: Help students visualize the input-output relationship
    """
    fig, ax = plt.subplots(figsize=(12, 4))

    # Plot the input sequence (last 12 months)
    ax.plot(range(SEQUENCE_LENGTH), X[seq_idx].flatten(), 'b-o',
            label=f'Input Sequence (t-{SEQUENCE_LENGTH} to t-1)',
            markersize=6, linewidth=2)

    # Plot the target value (next month)
    ax.plot(SEQUENCE_LENGTH, y[seq_idx], 'ro', markersize=12,
            label='Target (t)', zorder=5)

    ax.set_xlabel('Time Steps', fontsize=12)
    ax.set_ylabel('Scaled Value', fontsize=12)
    ax.set_title(f'Example Sequence {seq_idx}: How the Model "Sees" Data',
                 fontsize=14, fontweight='bold')
    ax.legend(fontsize=11)
    ax.grid(True, alpha=0.3)

    # Add annotations
    ax.annotate('Model predicts this\nfrom previous 12 values',
                xy=(SEQUENCE_LENGTH, y[seq_idx]),
                xytext=(SEQUENCE_LENGTH-3, y[seq_idx]+0.1),
                arrowprops=dict(arrowstyle='->', color='red', lw=2),
                fontsize=10, color='red', fontweight='bold')

    # Add shading to emphasize input vs output
    ax.axvspan(-0.5, SEQUENCE_LENGTH-0.5, alpha=0.1, color='blue',
               label='Input Window')
```

```
    plt.tight_layout()
    plt.show()

plot_sequence_example(X_train, y_train, seq_idx=0)

print("\n Key Insight:")
print("The model uses patterns in the blue region to predict the red point.")
print("It learns: 'Given this sequence pattern, what comes next?'")
```




```
 Key Insight:
The model uses patterns in the blue region to predict the red point.
It learns: 'Given this sequence pattern, what comes next?'
```

Participant Exercise:

1. Look at the plot - can you predict the next value by eye?
2. What patterns do you see in the input sequence?
3. Would you need more than 12 months to make a good prediction?

Comparison: RNN vs LSTM vs GRU

Feature	Simple RNN	LSTM	GRU
Parameters	Least	Most	Medium
Training Speed	Fastest	Slowest	Medium
Memory Capability	Short (10-20 steps)	Long (100+ steps)	Long (80+ steps)
Complexity	Simplest	Most complex	Moderate
Best For	Short sequences	Long dependencies	Balance of both

When to Use What?

- **Simple RNN**: Quick experiments, short sequences (<20), limited compute
- **LSTM**: Long sequences, complex patterns, production systems
- **GRU**: Slightly faster than LSTM, often performs similarly
- **Bidirectional**: When you can see future data (not for forecasting!)

5 Model Implementation

5.1 Building Deep Learning Models: General Strategy

Layer Stacking Philosophy:

1. **Input Layer**: Defines the data shape
2. **Recurrent Layers**: Extract temporal features (with return_sequences=True for stacking)
3. **Dropout Layers**: Prevent overfitting
4. **Dense Layers**: Learn complex combinations of features
5. **Output Layer**: Single neuron for regression

Design Principle: Start simple, add complexity only if needed!

5.2 Building the Simple RNN Model

We define a basic, stackable recurrent network (Simple RNN) to serve as a **baseline**. If more complex models don't beat this simple model, we know there's an issue with the data or setup.

5.2.1 Architecture

1. **Layer Stacking Philosophy:** We start with an input layer, add recurrent layers to extract temporal features, use Dropout for regularization, and finish with Dense layers for final prediction.
2. `SimpleRNN(50, return_sequences=True):` We choose 50 units as a balanced starting size. `return_sequences=True` is vital here because we are stacking another RNN layer on top, meaning we need the output for *every* timestep in the sequence, not just the last one.
3. `Dropout(0.2):` We randomly switch off 20% of neurons during training. This forces the network to learn more robust features and prevents over-reliance on any single neuron, combating overfitting.
4. `SimpleRNN(50, return_sequences=False):` For the last recurrent layer, we set `return_sequences=False` because we only need the final hidden state to generate our single output prediction for the next timestep.
5. `Dense(1):` A single output neuron is required because we are performing regression—predicting one numerical value (the next step).

5.2.2 Compilation

- **Optimizer (Adam):** Adam is efficient and adaptive, making it the most common and robust choice for most problems. We stick with the default **learning rate of 0.001**, which is generally a sweet spot. A learning rate that is too high causes unstable training; one that is too low means incredibly slow convergence.
- **Loss Function (MSE):** Mean Squared Error (MSE) is the standard loss function for regression tasks. It strongly penalizes large errors (squared error), encouraging highly accurate predictions.

5.2.3 Other Alternatives

- **Activation:** For the internal layers, you could use `Tanh` or `ReLU`. If using `StandardScaler`, `ReLU` works very well.
- **Loss: MAE (Mean Absolute Error)** is a viable alternative if you have high volatility or many outliers, as it is less sensitive than MSE.
- **Units:** You could try fewer units (32) for smaller datasets or more (64, 128) for larger, more complex ones.

5.2.4 Tips and Tricks

- **Overfitting:** If your training loss is much, much lower than your validation loss, your model is overfitting. Increase the dropout rate (e.g., 0.3 or 0.5) to introduce more regularization.
- **RNN Limitation:** Remember the key limitation: Simple RNNs struggle with **vanishing gradients** and cannot effectively learn dependencies that are more than 10-20 steps apart. This is why we move to LSTMs next.

[24]:
```
# Code Block 7: Building the Simple RNN Model
def create_rnn_model(input_shape, units=50, dropout_rate=0.2):
    """
    Create a simple RNN model

    Args:
        input_shape: tuple (timesteps, features) - e.g., (12, 1)
        units: number of RNN units in each layer
        dropout_rate: fraction of units to drop (0.0 to 1.0)

    Returns:
        compiled Keras model ready for training

    Architecture:
        RNN(50) → Dropout → RNN(50) → Dropout → Dense(25) → Dense(1)

      Tip: This is the simplest recurrent architecture
    """
    model = Sequential([
        # First RNN layer - extracts temporal patterns
        SimpleRNN(units, return_sequences=True, input_shape=input_shape,
                  activation='tanh'),  # tanh is default, ranges [-1, 1]

        # Dropout for regularization
        Dropout(dropout_rate),

        # Second RNN layer - learns higher-level patterns
        SimpleRNN(units, return_sequences=False),  # Only final output needed

        # More dropout
        Dropout(dropout_rate),
```

```
        # Dense layer for complex feature combinations
        Dense(25, activation='relu'),  # ReLU: max(0, x)

        # Output layer - single prediction
        Dense(1)  # No activation for regression
    ])

    # Compile model with optimizer and loss function
    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='mse',  # Mean Squared Error for regression
        metrics=['mae']  # Also track Mean Absolute Error
    )

    return model

# Create RNN model
print("Creating Simple RNN model...")
print("="*60)
rnn_model = create_rnn_model((SEQUENCE_LENGTH, 1), units=50)
print(rnn_model.summary())

print("\n Model Architecture Insights:")
print(f"• Total parameters: {rnn_model.count_params():,}")
print(f"• Input shape: ({SEQUENCE_LENGTH}, 1)")
print(f"• Output shape: (1,) - single value prediction")
print(f"• Memory usage: ~{rnn_model.count_params() * 4 / 1024:.2f} KB")
```

```
Creating Simple RNN model…
============================================================
Model: "sequential"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 12, 50)	2,600
dropout (Dropout)	(None, 12, 50)	0
simple_rnn_1 (SimpleRNN)	(None, 50)	5,050

```
 dropout_1 (Dropout)                  (None, 50)                                ␣
↪  0

 dense (Dense)                        (None, 25)                                ␣
↪1,275

 dense_1 (Dense)                      (None, 1)                                 ␣
↪ 26


 Total params: 8,951 (34.96 KB)


 Trainable params: 8,951 (34.96 KB)


 Non-trainable params: 0 (0.00 B)


None

 Model Architecture Insights:
• Total parameters: 8,951
• Input shape: (12, 1)
• Output shape: (1,) - single value prediction
• Memory usage: ~34.96 KB
```

5.3 Building the LSTM Model

We replace the `SimpleRNN` layers with `LSTM` layers. The purpose is to **mitigate the vanishing gradient problem** of the RNN and enable the model to capture long-term dependencies, such as complex annual seasonality.

We choose LSTMs because of their internal structure involving the **cell state** (the long-term memory conveyor belt) and **three specific gates** (Forget, Input, Output). These gates allow the network to selectively remember relevant past information and discard irrelevant information. This mechanism makes training much more stable for longer sequences.

5.3.1 Key Differences from RNN

- **Parameters:** LSTMs have approximately **4 times the parameters** of a Simple RNN because they must calculate weights for each of the three gates plus the cell state.
- **Memory Capability:** LSTMs can learn dependencies 100+ steps apart, far exceeding the capability of a Simple RNN.

5.3.2 Other Alternatives

- **GRU (Gated Recurrent Unit):** GRUs are similar to LSTMs but are slightly simpler and faster to train (they only have two gates). They offer a good balance between memory capability and speed.

5.3.3 Tips and Tricks

- **When to Use LSTM:** Always favor LSTM over Simple RNN if your sequence length is greater than 20 timesteps or if you know long-term patterns (like yearly seasonality) are crucial for the prediction.
- **Data Requirement:** Due to their higher parameter count, LSTMs generally require more training data than RNNs.

[26]:
```python
# Code Block 8: Building the LSTM Model
def create_lstm_model(input_shape, units=50, dropout_rate=0.2):
    """
    Create an LSTM model

    Args:
        input_shape: tuple (timesteps, features)
        units: number of LSTM units in each layer
        dropout_rate: dropout rate for regularization

    Returns:
        compiled Keras model

    Architecture:
        LSTM(50) → Dropout → LSTM(50) → Dropout → Dense(25) → Dense(1)

      Tip: LSTM has 4x more parameters than SimpleRNN!
    """
    model = Sequential([
        # First LSTM layer
        # return_sequences=True because we're stacking another LSTM
        LSTM(units, return_sequences=True, input_shape=input_shape,
             activation='tanh',  # tanh for gates
             recurrent_activation='sigmoid'),  # sigmoid for gates

        Dropout(dropout_rate),

        # Second LSTM layer
        # return_sequences=False because next layer is Dense
        LSTM(units, return_sequences=False),

        Dropout(dropout_rate),

        # Dense layers
        Dense(25, activation='relu'),
        Dense(1)
    ])

    model.compile(
        optimizer=Adam(learning_rate=0.001),
```

```
        loss='mse',
        metrics=['mae']
    )

    return model

# Create LSTM model
print("\nCreating LSTM model...")
print("="*60)
lstm_model = create_lstm_model((SEQUENCE_LENGTH, 1), units=50)
print(lstm_model.summary())

print("\n  LSTM vs RNN Parameter Count:")
print(f"• RNN parameters: {rnn_model.count_params():,}")
print(f"• LSTM parameters: {lstm_model.count_params():,}")
print(f"• Ratio: {lstm_model.count_params() / rnn_model.count_params():.1f}x␣
 ↪more")
print("\nWhy? LSTM has 4 sets of weights (forget, input, output, cell)")
```

```
Creating LSTM model…
============================================================
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 12, 50)	10,400
dropout_2 (Dropout)	(None, 12, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_3 (Dropout)	(None, 50)	0
dense_2 (Dense)	(None, 25)	1,275
dense_3 (Dense)	(None, 1)	26

```
 Total params: 31,901 (124.61 KB)


 Trainable params: 31,901 (124.61 KB)


 Non-trainable params: 0 (0.00 B)


None


 LSTM vs RNN Parameter Count:
• RNN parameters: 8,951
• LSTM parameters: 31,901
• Ratio: 3.6x more

Why? LSTM has 4 sets of weights (forget, input, output, cell)
```

LSTM Parameter Calculation:

```
params = 4 * [(input_dim + hidden_dim + 1) * hidden_dim]
       = 4 * [(1 + 50 + 1) * 50]
       = 4 * 2,600
       = 10,400 parameters
```

5.4 Advanced LSTM with Multiple Layers

This block defines a **deeper and more expressive LSTM architecture** for time series forecasting. It stacks **three LSTM layers** with **Dropout** between them to reduce overfitting and improve generalization. The model concludes with **Dense layers** to refine outputs before prediction. Using the **Adam optimizer** with a small learning rate ensures stable convergence. This setup captures **complex temporal dependencies** that simpler models might miss. Alternatives could include **GRU layers**, **bidirectional LSTMs**, or **attention-based models**, but this design balances depth, accuracy, and computational efficiency effectively.

[29]:
```python
# Code Block 9: Building the Advanced LSTM Model
def create_advanced_lstm_model(input_shape, units=50, dropout_rate=0.2):
    """
    Create an advanced LSTM model with multiple layers

    Args:
        input_shape: shape of input sequences
        units: number of LSTM units
        dropout_rate: dropout rate for regularization

    Returns:
        compiled Keras model
    """
    model = Sequential([
```

```
        LSTM(units, return_sequences=True, input_shape=input_shape),
        Dropout(dropout_rate),
        LSTM(units, return_sequences=True),
        Dropout(dropout_rate),
        LSTM(units, return_sequences=False),
        Dropout(dropout_rate),
        Dense(50),
        Dense(25),
        Dense(1)
    ])

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='mse',
        metrics=['mae']
    )

    return model

# Create advanced LSTM model
print("Creating advanced LSTM model...")
advanced_lstm_model = create_advanced_lstm_model((SEQUENCE_LENGTH, 1), units=50)
print(advanced_lstm_model.summary())
```

```
Creating advanced LSTM model...
Model: "sequential_2"
 Layer (type)                Output Shape              Param #
 lstm_2 (LSTM)               (None, 12, 50)            10,400
 dropout_4 (Dropout)         (None, 12, 50)            0
 lstm_3 (LSTM)               (None, 12, 50)            20,200
 dropout_5 (Dropout)         (None, 12, 50)            0
 lstm_4 (LSTM)               (None, 50)                20,200
```

```
dropout_6 (Dropout)                  (None, 50)                  ␣
↪  0

dense_4 (Dense)                      (None, 50)                  ␣
↪2,550

dense_5 (Dense)                      (None, 25)                  ␣
↪1,275

dense_6 (Dense)                      (None, 1)                   ␣
↪ 26


Total params: 54,651 (213.48 KB)

Trainable params: 54,651 (213.48 KB)

Non-trainable params: 0 (0.00 B)

None
```

6 Training and Evaluation

This code trains and evaluates three neural networks — a Simple RNN, an LSTM and an Advanced (stacked) LSTM — on the same dataset to compare their forecasting performance.

We define a reusable function `train_and_evaluate_model()` that handles everything: training, validation, prediction and error calculation.

6.1 Key Steps & Rationale

- **Training setup:** The model is trained with `epochs=100`, `batch_size=32`, and a `validation_split=0.2` to monitor generalization. These are balanced defaults — not too small to underfit, not too large to overfit.
- **Callbacks:**
  - `EarlyStopping` stops training if validation loss doesn't improve for 15 epochs — prevents overfitting.
  - `ReduceLROnPlateau` lowers the learning rate when progress stalls — helps reach finer minima. These make training adaptive and efficient.
- **Predictions & Scaling:** Predictions on training and test sets are inverse-transformed using the scaler to get results back to original units (e.g., export values).

- **Metrics:** We compute RMSE and MAE — two common measures of forecast accuracy. RMSE penalizes large errors; MAE is easier to interpret.

6.2 Alternatives and Tips

- Try **different sequence lengths** for time series instead of random splits.
- Use **ModelCheckpoint** to save best models automatically.
- Tune **batch size, learning rate** and **dropout** for better generalization.
- For small datasets, reduce epochs and validation split.

6.3 Why This Design Works

This approach ensures **fair, automated and interpretable model comparison** — each model is trained under the same conditions, evaluated on unseen data, and summarized with objective metrics. It's a clean, scalable workflow that balances accuracy with simplicity.

[31]:
```
# Code Block 10: Training the Model (`model.fit()`)
def train_and_evaluate_model(model, model_name, X_train, y_train, X_test,␣
 ↪y_test,
                             epochs=100, batch_size=32, validation_split=0.2):
    """
    Train and evaluate a model

    Args:
        model: Keras model to train
        model_name: name for display purposes
        X_train, y_train: training data
        X_test, y_test: test data
        epochs: number of training epochs
        batch_size: batch size for training
        validation_split: fraction of training data to use for validation

    Returns:
        trained model and training history
    """
    print(f"\n{'='*50}")
    print(f"Training {model_name}")
    print(f"{'='*50}")

    # Define callbacks
    early_stopping = tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=15,
        restore_best_weights=True
    )

    reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
```

```
        factor=0.5,
        patience=10,
        min_lr=0.0001
    )

    # Train the model
    history = model.fit(
        X_train, y_train,
        epochs=epochs,
        batch_size=batch_size,
        validation_split=validation_split,
        callbacks=[early_stopping, reduce_lr],
        verbose=1
    )

    # Make predictions
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    # Inverse transform predictions
    train_pred_inv = scaler.inverse_transform(train_pred)
    test_pred_inv = scaler.inverse_transform(test_pred)
    y_train_inv = scaler.inverse_transform(y_train.reshape(-1, 1))
    y_test_inv = scaler.inverse_transform(y_test.reshape(-1, 1))

    # Calculate metrics
    train_rmse = np.sqrt(mean_squared_error(y_train_inv, train_pred_inv))
    test_rmse = np.sqrt(mean_squared_error(y_test_inv, test_pred_inv))
    train_mae = mean_absolute_error(y_train_inv, train_pred_inv)
    test_mae = mean_absolute_error(y_test_inv, test_pred_inv)

    print(f"\n{model_name} Performance:")
    print(f"Training RMSE: {train_rmse:.4f}")
    print(f"Test RMSE: {test_rmse:.4f}")
    print(f"Training MAE: {train_mae:.4f}")
    print(f"Test MAE: {test_mae:.4f}")

    return model, history, {
        'train_pred': train_pred_inv,
        'test_pred': test_pred_inv,
        'train_actual': y_train_inv,
        'test_actual': y_test_inv,
        'train_rmse': train_rmse,
        'test_rmse': test_rmse,
        'train_mae': train_mae,
        'test_mae': test_mae
    }
```

```
# Train all models
models_results = {}

# Train RNN
rnn_model, rnn_history, rnn_results = train_and_evaluate_model(
    rnn_model, "Simple RNN", X_train, y_train, X_test, y_test
)
models_results['RNN'] = rnn_results

# Train LSTM
lstm_model, lstm_history, lstm_results = train_and_evaluate_model(
    lstm_model, "LSTM", X_train, y_train, X_test, y_test
)
models_results['LSTM'] = lstm_results

# Train Advanced LSTM
advanced_lstm_model, advanced_lstm_history, advanced_lstm_results =␣
 ↪train_and_evaluate_model(
    advanced_lstm_model, "Advanced LSTM", X_train, y_train, X_test, y_test
)
models_results['Advanced LSTM'] = advanced_lstm_results
```

```
==================================================
Training Simple RNN
==================================================
Epoch 1/100
6/6             2s 76ms/step - loss:
0.0893 - mae: 0.2216 - val_loss: 0.0162 - val_mae: 0.1087 - learning_rate:
0.0010
Epoch 2/100
6/6             0s 18ms/step - loss:
0.0507 - mae: 0.1653 - val_loss: 0.0430 - val_mae: 0.1755 - learning_rate:
0.0010
Epoch 3/100
6/6             0s 21ms/step - loss:
0.0422 - mae: 0.1577 - val_loss: 0.0290 - val_mae: 0.1442 - learning_rate:
0.0010
Epoch 4/100
6/6             0s 23ms/step - loss:
0.0315 - mae: 0.1217 - val_loss: 0.0320 - val_mae: 0.1453 - learning_rate:
0.0010
Epoch 5/100
6/6             0s 20ms/step - loss:
0.0201 - mae: 0.0999 - val_loss: 0.0169 - val_mae: 0.1021 - learning_rate:
0.0010
```

```
Epoch 6/100
6/6             0s 21ms/step - loss:
0.0182 - mae: 0.1017 - val_loss: 0.0114 - val_mae: 0.0800 - learning_rate:
0.0010
Epoch 7/100
6/6             0s 20ms/step - loss:
0.0167 - mae: 0.0914 - val_loss: 0.0130 - val_mae: 0.0855 - learning_rate:
0.0010
Epoch 8/100
6/6             0s 21ms/step - loss:
0.0142 - mae: 0.0834 - val_loss: 0.0184 - val_mae: 0.1066 - learning_rate:
0.0010
Epoch 9/100
6/6             0s 21ms/step - loss:
0.0171 - mae: 0.0936 - val_loss: 0.0123 - val_mae: 0.0813 - learning_rate:
0.0010
Epoch 10/100
6/6             0s 18ms/step - loss:
0.0160 - mae: 0.0887 - val_loss: 0.0100 - val_mae: 0.0789 - learning_rate:
0.0010
Epoch 11/100
6/6             0s 23ms/step - loss:
0.0164 - mae: 0.0938 - val_loss: 0.0101 - val_mae: 0.0763 - learning_rate:
0.0010
Epoch 12/100
6/6             0s 20ms/step - loss:
0.0138 - mae: 0.0786 - val_loss: 0.0110 - val_mae: 0.0776 - learning_rate:
0.0010
Epoch 13/100
6/6             0s 22ms/step - loss:
0.0147 - mae: 0.0850 - val_loss: 0.0101 - val_mae: 0.0793 - learning_rate:
0.0010
Epoch 14/100
6/6             0s 17ms/step - loss:
0.0156 - mae: 0.0900 - val_loss: 0.0114 - val_mae: 0.0813 - learning_rate:
0.0010
Epoch 15/100
6/6             0s 19ms/step - loss:
0.0163 - mae: 0.0837 - val_loss: 0.0135 - val_mae: 0.0875 - learning_rate:
0.0010
Epoch 16/100
6/6             0s 20ms/step - loss:
0.0136 - mae: 0.0776 - val_loss: 0.0119 - val_mae: 0.0898 - learning_rate:
0.0010
Epoch 17/100
6/6             0s 20ms/step - loss:
0.0139 - mae: 0.0803 - val_loss: 0.0104 - val_mae: 0.0776 - learning_rate:
0.0010
```

```
Epoch 18/100
6/6             0s 23ms/step - loss:
0.0135 - mae: 0.0800 - val_loss: 0.0115 - val_mae: 0.0805 - learning_rate:
0.0010
Epoch 19/100
6/6             0s 20ms/step - loss:
0.0139 - mae: 0.0804 - val_loss: 0.0091 - val_mae: 0.0754 - learning_rate:
0.0010
Epoch 20/100
6/6             0s 18ms/step - loss:
0.0129 - mae: 0.0826 - val_loss: 0.0092 - val_mae: 0.0752 - learning_rate:
0.0010
Epoch 21/100
6/6             0s 17ms/step - loss:
0.0122 - mae: 0.0788 - val_loss: 0.0106 - val_mae: 0.0768 - learning_rate:
0.0010
Epoch 22/100
6/6             0s 18ms/step - loss:
0.0131 - mae: 0.0784 - val_loss: 0.0097 - val_mae: 0.0754 - learning_rate:
0.0010
Epoch 23/100
6/6             0s 16ms/step - loss:
0.0115 - mae: 0.0786 - val_loss: 0.0096 - val_mae: 0.0740 - learning_rate:
0.0010
Epoch 24/100
6/6             0s 18ms/step - loss:
0.0145 - mae: 0.0826 - val_loss: 0.0108 - val_mae: 0.0773 - learning_rate:
0.0010
Epoch 25/100
6/6             0s 16ms/step - loss:
0.0131 - mae: 0.0783 - val_loss: 0.0095 - val_mae: 0.0763 - learning_rate:
0.0010
Epoch 26/100
6/6             0s 19ms/step - loss:
0.0138 - mae: 0.0808 - val_loss: 0.0092 - val_mae: 0.0751 - learning_rate:
0.0010
Epoch 27/100
6/6             0s 18ms/step - loss:
0.0137 - mae: 0.0821 - val_loss: 0.0092 - val_mae: 0.0734 - learning_rate:
0.0010
Epoch 28/100
6/6             0s 20ms/step - loss:
0.0121 - mae: 0.0785 - val_loss: 0.0091 - val_mae: 0.0732 - learning_rate:
0.0010
Epoch 29/100
6/6             0s 17ms/step - loss:
0.0124 - mae: 0.0789 - val_loss: 0.0091 - val_mae: 0.0729 - learning_rate:
0.0010
```

```
Epoch 30/100
6/6             0s 23ms/step - loss:
0.0110 - mae: 0.0736 - val_loss: 0.0087 - val_mae: 0.0742 - learning_rate:
5.0000e-04
Epoch 31/100
6/6             0s 18ms/step - loss:
0.0132 - mae: 0.0826 - val_loss: 0.0088 - val_mae: 0.0729 - learning_rate:
5.0000e-04
Epoch 32/100
6/6             0s 17ms/step - loss:
0.0131 - mae: 0.0771 - val_loss: 0.0095 - val_mae: 0.0727 - learning_rate:
5.0000e-04
Epoch 33/100
6/6             0s 17ms/step - loss:
0.0132 - mae: 0.0780 - val_loss: 0.0093 - val_mae: 0.0729 - learning_rate:
5.0000e-04
Epoch 34/100
6/6             0s 21ms/step - loss:
0.0124 - mae: 0.0744 - val_loss: 0.0092 - val_mae: 0.0757 - learning_rate:
5.0000e-04
Epoch 35/100
6/6             0s 19ms/step - loss:
0.0116 - mae: 0.0746 - val_loss: 0.0096 - val_mae: 0.0759 - learning_rate:
5.0000e-04
Epoch 36/100
6/6             0s 18ms/step - loss:
0.0114 - mae: 0.0747 - val_loss: 0.0102 - val_mae: 0.0753 - learning_rate:
5.0000e-04
Epoch 37/100
6/6             0s 19ms/step - loss:
0.0113 - mae: 0.0718 - val_loss: 0.0104 - val_mae: 0.0756 - learning_rate:
5.0000e-04
Epoch 38/100
6/6             0s 17ms/step - loss:
0.0121 - mae: 0.0705 - val_loss: 0.0095 - val_mae: 0.0736 - learning_rate:
5.0000e-04
Epoch 39/100
6/6             0s 16ms/step - loss:
0.0118 - mae: 0.0728 - val_loss: 0.0089 - val_mae: 0.0749 - learning_rate:
5.0000e-04
Epoch 40/100
6/6             0s 18ms/step - loss:
0.0121 - mae: 0.0802 - val_loss: 0.0090 - val_mae: 0.0743 - learning_rate:
5.0000e-04
Epoch 41/100
6/6             0s 19ms/step - loss:
0.0124 - mae: 0.0776 - val_loss: 0.0092 - val_mae: 0.0740 - learning_rate:
2.5000e-04
```

```
Epoch 42/100
6/6             0s 19ms/step - loss:
0.0119 - mae: 0.0734 - val_loss: 0.0092 - val_mae: 0.0741 - learning_rate:
2.5000e-04
Epoch 43/100
6/6             0s 19ms/step - loss:
0.0111 - mae: 0.0694 - val_loss: 0.0091 - val_mae: 0.0742 - learning_rate:
2.5000e-04
Epoch 44/100
6/6             0s 17ms/step - loss:
0.0101 - mae: 0.0731 - val_loss: 0.0091 - val_mae: 0.0739 - learning_rate:
2.5000e-04
Epoch 45/100
6/6             0s 19ms/step - loss:
0.0098 - mae: 0.0712 - val_loss: 0.0091 - val_mae: 0.0741 - learning_rate:
2.5000e-04
7/7             0s 33ms/step
2/2             0s 27ms/step

Simple RNN Performance:
Training RMSE: 53.2121
Test RMSE: 71.9380
Training MAE: 36.7714
Test MAE: 53.7698

================================================
Training LSTM
================================================
Epoch 1/100
6/6             2s 78ms/step - loss:
0.0499 - mae: 0.1731 - val_loss: 0.0133 - val_mae: 0.0852 - learning_rate:
0.0010
Epoch 2/100
6/6             0s 17ms/step - loss:
0.0163 - mae: 0.0973 - val_loss: 0.0132 - val_mae: 0.0960 - learning_rate:
0.0010
Epoch 3/100
6/6             0s 19ms/step - loss:
0.0146 - mae: 0.0893 - val_loss: 0.0146 - val_mae: 0.0904 - learning_rate:
0.0010
Epoch 4/100
6/6             0s 30ms/step - loss:
0.0130 - mae: 0.0737 - val_loss: 0.0135 - val_mae: 0.0862 - learning_rate:
0.0010
Epoch 5/100
6/6             0s 23ms/step - loss:
0.0117 - mae: 0.0732 - val_loss: 0.0120 - val_mae: 0.0862 - learning_rate:
0.0010
```

```
Epoch 6/100
6/6             0s 22ms/step - loss:
0.0125 - mae: 0.0724 - val_loss: 0.0121 - val_mae: 0.0834 - learning_rate:
0.0010
Epoch 7/100
6/6             0s 22ms/step - loss:
0.0109 - mae: 0.0674 - val_loss: 0.0124 - val_mae: 0.0839 - learning_rate:
0.0010
Epoch 8/100
6/6             0s 22ms/step - loss:
0.0123 - mae: 0.0739 - val_loss: 0.0124 - val_mae: 0.0867 - learning_rate:
0.0010
Epoch 9/100
6/6             0s 28ms/step - loss:
0.0129 - mae: 0.0749 - val_loss: 0.0123 - val_mae: 0.0848 - learning_rate:
0.0010
Epoch 10/100
6/6             0s 23ms/step - loss:
0.0122 - mae: 0.0732 - val_loss: 0.0123 - val_mae: 0.0856 - learning_rate:
0.0010
Epoch 11/100
6/6             0s 22ms/step - loss:
0.0117 - mae: 0.0715 - val_loss: 0.0122 - val_mae: 0.0850 - learning_rate:
0.0010
Epoch 12/100
6/6             0s 21ms/step - loss:
0.0119 - mae: 0.0733 - val_loss: 0.0121 - val_mae: 0.0859 - learning_rate:
0.0010
Epoch 13/100
6/6             0s 25ms/step - loss:
0.0126 - mae: 0.0711 - val_loss: 0.0121 - val_mae: 0.0837 - learning_rate:
0.0010
Epoch 14/100
6/6             0s 20ms/step - loss:
0.0120 - mae: 0.0720 - val_loss: 0.0120 - val_mae: 0.0852 - learning_rate:
0.0010
Epoch 15/100
6/6             0s 21ms/step - loss:
0.0117 - mae: 0.0732 - val_loss: 0.0120 - val_mae: 0.0868 - learning_rate:
0.0010
Epoch 16/100
6/6             0s 22ms/step - loss:
0.0112 - mae: 0.0726 - val_loss: 0.0119 - val_mae: 0.0853 - learning_rate:
5.0000e-04
Epoch 17/100
6/6             0s 20ms/step - loss:
0.0111 - mae: 0.0680 - val_loss: 0.0119 - val_mae: 0.0854 - learning_rate:
5.0000e-04
```

```
Epoch 18/100
6/6             0s 23ms/step - loss:
0.0119 - mae: 0.0728 - val_loss: 0.0120 - val_mae: 0.0869 - learning_rate:
5.0000e-04
Epoch 19/100
6/6             0s 22ms/step - loss:
0.0115 - mae: 0.0697 - val_loss: 0.0119 - val_mae: 0.0853 - learning_rate:
5.0000e-04
Epoch 20/100
6/6             0s 23ms/step - loss:
0.0120 - mae: 0.0717 - val_loss: 0.0118 - val_mae: 0.0841 - learning_rate:
5.0000e-04
Epoch 21/100
6/6             0s 27ms/step - loss:
0.0116 - mae: 0.0719 - val_loss: 0.0118 - val_mae: 0.0835 - learning_rate:
5.0000e-04
Epoch 22/100
6/6             0s 23ms/step - loss:
0.0114 - mae: 0.0699 - val_loss: 0.0118 - val_mae: 0.0847 - learning_rate:
5.0000e-04
Epoch 23/100
6/6             0s 26ms/step - loss:
0.0109 - mae: 0.0699 - val_loss: 0.0118 - val_mae: 0.0857 - learning_rate:
5.0000e-04
Epoch 24/100
6/6             0s 24ms/step - loss:
0.0112 - mae: 0.0697 - val_loss: 0.0118 - val_mae: 0.0851 - learning_rate:
5.0000e-04
Epoch 25/100
6/6             0s 30ms/step - loss:
0.0115 - mae: 0.0697 - val_loss: 0.0118 - val_mae: 0.0829 - learning_rate:
5.0000e-04
Epoch 26/100
6/6             0s 26ms/step - loss:
0.0116 - mae: 0.0689 - val_loss: 0.0117 - val_mae: 0.0852 - learning_rate:
5.0000e-04
Epoch 27/100
6/6             0s 30ms/step - loss:
0.0108 - mae: 0.0694 - val_loss: 0.0117 - val_mae: 0.0861 - learning_rate:
5.0000e-04
Epoch 28/100
6/6             0s 32ms/step - loss:
0.0125 - mae: 0.0754 - val_loss: 0.0117 - val_mae: 0.0831 - learning_rate:
5.0000e-04
Epoch 29/100
6/6             0s 27ms/step - loss:
0.0114 - mae: 0.0688 - val_loss: 0.0116 - val_mae: 0.0844 - learning_rate:
5.0000e-04
```

```
Epoch 30/100
6/6             0s 27ms/step - loss:
0.0124 - mae: 0.0728 - val_loss: 0.0116 - val_mae: 0.0844 - learning_rate:
5.0000e-04
Epoch 31/100
6/6             0s 29ms/step - loss:
0.0113 - mae: 0.0698 - val_loss: 0.0116 - val_mae: 0.0834 - learning_rate:
5.0000e-04
Epoch 32/100
6/6             0s 24ms/step - loss:
0.0115 - mae: 0.0728 - val_loss: 0.0116 - val_mae: 0.0833 - learning_rate:
5.0000e-04
Epoch 33/100
6/6             0s 25ms/step - loss:
0.0117 - mae: 0.0701 - val_loss: 0.0116 - val_mae: 0.0842 - learning_rate:
5.0000e-04
Epoch 34/100
6/6             0s 24ms/step - loss:
0.0115 - mae: 0.0705 - val_loss: 0.0116 - val_mae: 0.0837 - learning_rate:
5.0000e-04
Epoch 35/100
6/6             0s 28ms/step - loss:
0.0111 - mae: 0.0692 - val_loss: 0.0116 - val_mae: 0.0846 - learning_rate:
5.0000e-04
Epoch 36/100
6/6             0s 28ms/step - loss:
0.0127 - mae: 0.0740 - val_loss: 0.0116 - val_mae: 0.0853 - learning_rate:
5.0000e-04
Epoch 37/100
6/6             0s 28ms/step - loss:
0.0123 - mae: 0.0727 - val_loss: 0.0115 - val_mae: 0.0833 - learning_rate:
5.0000e-04
Epoch 38/100
6/6             0s 25ms/step - loss:
0.0119 - mae: 0.0717 - val_loss: 0.0115 - val_mae: 0.0837 - learning_rate:
5.0000e-04
Epoch 39/100
6/6             0s 30ms/step - loss:
0.0120 - mae: 0.0725 - val_loss: 0.0115 - val_mae: 0.0837 - learning_rate:
5.0000e-04
Epoch 40/100
6/6             0s 30ms/step - loss:
0.0112 - mae: 0.0699 - val_loss: 0.0115 - val_mae: 0.0839 - learning_rate:
5.0000e-04
Epoch 41/100
6/6             0s 30ms/step - loss:
0.0114 - mae: 0.0702 - val_loss: 0.0115 - val_mae: 0.0833 - learning_rate:
5.0000e-04
```

```
Epoch 42/100
6/6             0s 28ms/step - loss:
0.0110 - mae: 0.0697 - val_loss: 0.0115 - val_mae: 0.0842 - learning_rate:
5.0000e-04
Epoch 43/100
6/6             0s 28ms/step - loss:
0.0109 - mae: 0.0672 - val_loss: 0.0115 - val_mae: 0.0839 - learning_rate:
5.0000e-04
Epoch 44/100
6/6             0s 28ms/step - loss:
0.0111 - mae: 0.0703 - val_loss: 0.0114 - val_mae: 0.0837 - learning_rate:
5.0000e-04
Epoch 45/100
6/6             0s 25ms/step - loss:
0.0117 - mae: 0.0711 - val_loss: 0.0114 - val_mae: 0.0834 - learning_rate:
5.0000e-04
Epoch 46/100
6/6             0s 26ms/step - loss:
0.0108 - mae: 0.0682 - val_loss: 0.0114 - val_mae: 0.0836 - learning_rate:
5.0000e-04
Epoch 47/100
6/6             0s 23ms/step - loss:
0.0117 - mae: 0.0721 - val_loss: 0.0113 - val_mae: 0.0832 - learning_rate:
5.0000e-04
Epoch 48/100
6/6             0s 24ms/step - loss:
0.0110 - mae: 0.0674 - val_loss: 0.0113 - val_mae: 0.0835 - learning_rate:
5.0000e-04
Epoch 49/100
6/6             0s 25ms/step - loss:
0.0116 - mae: 0.0708 - val_loss: 0.0113 - val_mae: 0.0826 - learning_rate:
5.0000e-04
Epoch 50/100
6/6             0s 28ms/step - loss:
0.0114 - mae: 0.0677 - val_loss: 0.0113 - val_mae: 0.0837 - learning_rate:
5.0000e-04
Epoch 51/100
6/6             0s 25ms/step - loss:
0.0113 - mae: 0.0707 - val_loss: 0.0112 - val_mae: 0.0835 - learning_rate:
5.0000e-04
Epoch 52/100
6/6             0s 27ms/step - loss:
0.0111 - mae: 0.0701 - val_loss: 0.0112 - val_mae: 0.0836 - learning_rate:
5.0000e-04
Epoch 53/100
6/6             0s 27ms/step - loss:
0.0107 - mae: 0.0670 - val_loss: 0.0112 - val_mae: 0.0844 - learning_rate:
5.0000e-04
```

```
Epoch 54/100
6/6             0s 28ms/step - loss:
0.0113 - mae: 0.0697 - val_loss: 0.0112 - val_mae: 0.0833 - learning_rate:
5.0000e-04
Epoch 55/100
6/6             0s 30ms/step - loss:
0.0111 - mae: 0.0715 - val_loss: 0.0112 - val_mae: 0.0828 - learning_rate:
5.0000e-04
Epoch 56/100
6/6             0s 23ms/step - loss:
0.0112 - mae: 0.0698 - val_loss: 0.0112 - val_mae: 0.0839 - learning_rate:
5.0000e-04
Epoch 57/100
6/6             0s 19ms/step - loss:
0.0114 - mae: 0.0715 - val_loss: 0.0112 - val_mae: 0.0839 - learning_rate:
5.0000e-04
Epoch 58/100
6/6             0s 23ms/step - loss:
0.0114 - mae: 0.0702 - val_loss: 0.0112 - val_mae: 0.0840 - learning_rate:
5.0000e-04
Epoch 59/100
6/6             0s 23ms/step - loss:
0.0115 - mae: 0.0713 - val_loss: 0.0112 - val_mae: 0.0847 - learning_rate:
5.0000e-04
Epoch 60/100
6/6             0s 23ms/step - loss:
0.0113 - mae: 0.0693 - val_loss: 0.0111 - val_mae: 0.0827 - learning_rate:
5.0000e-04
Epoch 61/100
6/6             0s 24ms/step - loss:
0.0109 - mae: 0.0695 - val_loss: 0.0111 - val_mae: 0.0832 - learning_rate:
5.0000e-04
Epoch 62/100
6/6             0s 23ms/step - loss:
0.0111 - mae: 0.0703 - val_loss: 0.0111 - val_mae: 0.0843 - learning_rate:
5.0000e-04
Epoch 63/100
6/6             0s 27ms/step - loss:
0.0112 - mae: 0.0689 - val_loss: 0.0111 - val_mae: 0.0832 - learning_rate:
5.0000e-04
Epoch 64/100
6/6             0s 26ms/step - loss:
0.0116 - mae: 0.0725 - val_loss: 0.0111 - val_mae: 0.0834 - learning_rate:
5.0000e-04
Epoch 65/100
6/6             0s 30ms/step - loss:
0.0099 - mae: 0.0666 - val_loss: 0.0111 - val_mae: 0.0849 - learning_rate:
5.0000e-04
```

```
Epoch 66/100
6/6             0s 30ms/step - loss:
0.0115 - mae: 0.0711 - val_loss: 0.0111 - val_mae: 0.0838 - learning_rate:
5.0000e-04
Epoch 67/100
6/6             0s 29ms/step - loss:
0.0110 - mae: 0.0688 - val_loss: 0.0110 - val_mae: 0.0842 - learning_rate:
5.0000e-04
Epoch 68/100
6/6             0s 26ms/step - loss:
0.0118 - mae: 0.0730 - val_loss: 0.0110 - val_mae: 0.0839 - learning_rate:
5.0000e-04
Epoch 69/100
6/6             0s 23ms/step - loss:
0.0113 - mae: 0.0713 - val_loss: 0.0110 - val_mae: 0.0845 - learning_rate:
5.0000e-04
Epoch 70/100
6/6             0s 23ms/step - loss:
0.0111 - mae: 0.0687 - val_loss: 0.0111 - val_mae: 0.0850 - learning_rate:
5.0000e-04
Epoch 71/100
6/6             0s 23ms/step - loss:
0.0109 - mae: 0.0698 - val_loss: 0.0110 - val_mae: 0.0847 - learning_rate:
2.5000e-04
Epoch 72/100
6/6             0s 24ms/step - loss:
0.0116 - mae: 0.0722 - val_loss: 0.0110 - val_mae: 0.0838 - learning_rate:
2.5000e-04
Epoch 73/100
6/6             0s 24ms/step - loss:
0.0114 - mae: 0.0713 - val_loss: 0.0109 - val_mae: 0.0835 - learning_rate:
2.5000e-04
Epoch 74/100
6/6             0s 28ms/step - loss:
0.0112 - mae: 0.0707 - val_loss: 0.0109 - val_mae: 0.0838 - learning_rate:
2.5000e-04
Epoch 75/100
6/6             0s 24ms/step - loss:
0.0113 - mae: 0.0728 - val_loss: 0.0109 - val_mae: 0.0844 - learning_rate:
2.5000e-04
Epoch 76/100
6/6             0s 25ms/step - loss:
0.0107 - mae: 0.0706 - val_loss: 0.0109 - val_mae: 0.0843 - learning_rate:
2.5000e-04
Epoch 77/100
6/6             0s 24ms/step - loss:
0.0113 - mae: 0.0699 - val_loss: 0.0108 - val_mae: 0.0834 - learning_rate:
2.5000e-04
```

```
Epoch 78/100
6/6             0s 33ms/step - loss:
0.0110 - mae: 0.0680 - val_loss: 0.0108 - val_mae: 0.0836 - learning_rate:
2.5000e-04
Epoch 79/100
6/6             0s 25ms/step - loss:
0.0113 - mae: 0.0703 - val_loss: 0.0108 - val_mae: 0.0833 - learning_rate:
2.5000e-04
Epoch 80/100
6/6             0s 29ms/step - loss:
0.0112 - mae: 0.0696 - val_loss: 0.0109 - val_mae: 0.0839 - learning_rate:
2.5000e-04
Epoch 81/100
6/6             0s 26ms/step - loss:
0.0113 - mae: 0.0700 - val_loss: 0.0110 - val_mae: 0.0849 - learning_rate:
2.5000e-04
Epoch 82/100
6/6             0s 30ms/step - loss:
0.0107 - mae: 0.0679 - val_loss: 0.0110 - val_mae: 0.0851 - learning_rate:
2.5000e-04
Epoch 83/100
6/6             0s 27ms/step - loss:
0.0107 - mae: 0.0689 - val_loss: 0.0110 - val_mae: 0.0847 - learning_rate:
2.5000e-04
Epoch 84/100
6/6             0s 27ms/step - loss:
0.0110 - mae: 0.0692 - val_loss: 0.0110 - val_mae: 0.0846 - learning_rate:
2.5000e-04
Epoch 85/100
6/6             0s 26ms/step - loss:
0.0109 - mae: 0.0689 - val_loss: 0.0109 - val_mae: 0.0845 - learning_rate:
2.5000e-04
Epoch 86/100
6/6             0s 29ms/step - loss:
0.0111 - mae: 0.0698 - val_loss: 0.0109 - val_mae: 0.0841 - learning_rate:
2.5000e-04
Epoch 87/100
6/6             0s 24ms/step - loss:
0.0107 - mae: 0.0679 - val_loss: 0.0110 - val_mae: 0.0846 - learning_rate:
2.5000e-04
Epoch 88/100
6/6             0s 30ms/step - loss:
0.0108 - mae: 0.0675 - val_loss: 0.0110 - val_mae: 0.0847 - learning_rate:
1.2500e-04
Epoch 89/100
6/6             0s 23ms/step - loss:
0.0110 - mae: 0.0696 - val_loss: 0.0110 - val_mae: 0.0847 - learning_rate:
1.2500e-04
```

```
Epoch 90/100
6/6             0s 26ms/step - loss:
0.0110 - mae: 0.0715 - val_loss: 0.0110 - val_mae: 0.0847 - learning_rate:
1.2500e-04
Epoch 91/100
6/6             0s 27ms/step - loss:
0.0113 - mae: 0.0710 - val_loss: 0.0109 - val_mae: 0.0841 - learning_rate:
1.2500e-04
Epoch 92/100
6/6             0s 25ms/step - loss:
0.0114 - mae: 0.0717 - val_loss: 0.0109 - val_mae: 0.0837 - learning_rate:
1.2500e-04
Epoch 93/100
6/6             0s 22ms/step - loss:
0.0111 - mae: 0.0705 - val_loss: 0.0109 - val_mae: 0.0838 - learning_rate:
1.2500e-04
Epoch 94/100
6/6             0s 25ms/step - loss:
0.0109 - mae: 0.0694 - val_loss: 0.0109 - val_mae: 0.0839 - learning_rate:
1.2500e-04
7/7             1s 44ms/step
2/2             0s 22ms/step

LSTM Performance:
Training RMSE: 55.1235
Test RMSE: 67.7839
Training MAE: 37.9461
Test MAE: 49.1420

================================================
Training Advanced LSTM
================================================
Epoch 1/100
6/6             4s 127ms/step - loss:
0.0490 - mae: 0.1669 - val_loss: 0.0169 - val_mae: 0.1143 - learning_rate:
0.0010
Epoch 2/100
6/6             0s 30ms/step - loss:
0.0197 - mae: 0.1152 - val_loss: 0.0177 - val_mae: 0.1021 - learning_rate:
0.0010
Epoch 3/100
6/6             0s 25ms/step - loss:
0.0171 - mae: 0.0859 - val_loss: 0.0135 - val_mae: 0.0863 - learning_rate:
0.0010
Epoch 4/100
6/6             0s 27ms/step - loss:
0.0130 - mae: 0.0787 - val_loss: 0.0129 - val_mae: 0.0920 - learning_rate:
0.0010
```

```
Epoch 5/100
6/6             0s 26ms/step - loss:
0.0126 - mae: 0.0783 - val_loss: 0.0138 - val_mae: 0.0879 - learning_rate:
0.0010
Epoch 6/100
6/6             0s 34ms/step - loss:
0.0124 - mae: 0.0721 - val_loss: 0.0130 - val_mae: 0.0887 - learning_rate:
0.0010
Epoch 7/100
6/6             0s 26ms/step - loss:
0.0117 - mae: 0.0728 - val_loss: 0.0128 - val_mae: 0.0854 - learning_rate:
0.0010
Epoch 8/100
6/6             0s 28ms/step - loss:
0.0121 - mae: 0.0709 - val_loss: 0.0126 - val_mae: 0.0838 - learning_rate:
0.0010
Epoch 9/100
6/6             0s 27ms/step - loss:
0.0121 - mae: 0.0717 - val_loss: 0.0125 - val_mae: 0.0850 - learning_rate:
0.0010
Epoch 10/100
6/6             0s 29ms/step - loss:
0.0119 - mae: 0.0711 - val_loss: 0.0125 - val_mae: 0.0833 - learning_rate:
0.0010
Epoch 11/100
6/6             0s 27ms/step - loss:
0.0122 - mae: 0.0736 - val_loss: 0.0126 - val_mae: 0.0829 - learning_rate:
0.0010
Epoch 12/100
6/6             0s 29ms/step - loss:
0.0122 - mae: 0.0721 - val_loss: 0.0125 - val_mae: 0.0851 - learning_rate:
0.0010
Epoch 13/100
6/6             0s 30ms/step - loss:
0.0119 - mae: 0.0717 - val_loss: 0.0133 - val_mae: 0.0924 - learning_rate:
0.0010
Epoch 14/100
6/6             0s 26ms/step - loss:
0.0123 - mae: 0.0758 - val_loss: 0.0129 - val_mae: 0.0842 - learning_rate:
0.0010
Epoch 15/100
6/6             0s 32ms/step - loss:
0.0121 - mae: 0.0704 - val_loss: 0.0127 - val_mae: 0.0883 - learning_rate:
0.0010
Epoch 16/100
6/6             0s 28ms/step - loss:
0.0120 - mae: 0.0742 - val_loss: 0.0126 - val_mae: 0.0832 - learning_rate:
0.0010
```

```
Epoch 17/100
6/6             0s 23ms/step - loss:
0.0121 - mae: 0.0698 - val_loss: 0.0123 - val_mae: 0.0844 - learning_rate:
0.0010
Epoch 18/100
6/6             0s 27ms/step - loss:
0.0113 - mae: 0.0709 - val_loss: 0.0124 - val_mae: 0.0860 - learning_rate:
0.0010
Epoch 19/100
6/6             0s 35ms/step - loss:
0.0129 - mae: 0.0761 - val_loss: 0.0125 - val_mae: 0.0841 - learning_rate:
0.0010
Epoch 20/100
6/6             0s 30ms/step - loss:
0.0122 - mae: 0.0717 - val_loss: 0.0126 - val_mae: 0.0878 - learning_rate:
0.0010
Epoch 21/100
6/6             0s 33ms/step - loss:
0.0127 - mae: 0.0752 - val_loss: 0.0125 - val_mae: 0.0834 - learning_rate:
0.0010
Epoch 22/100
6/6             0s 28ms/step - loss:
0.0121 - mae: 0.0728 - val_loss: 0.0129 - val_mae: 0.0907 - learning_rate:
0.0010
Epoch 23/100
6/6             0s 31ms/step - loss:
0.0125 - mae: 0.0761 - val_loss: 0.0126 - val_mae: 0.0835 - learning_rate:
0.0010
Epoch 24/100
6/6             0s 27ms/step - loss:
0.0125 - mae: 0.0725 - val_loss: 0.0125 - val_mae: 0.0879 - learning_rate:
0.0010
Epoch 25/100
6/6             0s 29ms/step - loss:
0.0123 - mae: 0.0735 - val_loss: 0.0123 - val_mae: 0.0838 - learning_rate:
0.0010
Epoch 26/100
6/6             0s 26ms/step - loss:
0.0117 - mae: 0.0686 - val_loss: 0.0124 - val_mae: 0.0869 - learning_rate:
0.0010
Epoch 27/100
6/6             0s 25ms/step - loss:
0.0116 - mae: 0.0729 - val_loss: 0.0124 - val_mae: 0.0856 - learning_rate:
0.0010
Epoch 28/100
6/6             0s 32ms/step - loss:
0.0117 - mae: 0.0711 - val_loss: 0.0124 - val_mae: 0.0837 - learning_rate:
5.0000e-04
```

```
Epoch 29/100
6/6             0s 33ms/step - loss:
0.0115 - mae: 0.0695 - val_loss: 0.0123 - val_mae: 0.0866 - learning_rate:
5.0000e-04
Epoch 30/100
6/6             0s 38ms/step - loss:
0.0117 - mae: 0.0724 - val_loss: 0.0123 - val_mae: 0.0857 - learning_rate:
5.0000e-04
Epoch 31/100
6/6             0s 31ms/step - loss:
0.0121 - mae: 0.0729 - val_loss: 0.0123 - val_mae: 0.0841 - learning_rate:
5.0000e-04
Epoch 32/100
6/6             0s 31ms/step - loss:
0.0110 - mae: 0.0702 - val_loss: 0.0123 - val_mae: 0.0865 - learning_rate:
5.0000e-04
Epoch 33/100
6/6             0s 35ms/step - loss:
0.0117 - mae: 0.0727 - val_loss: 0.0124 - val_mae: 0.0867 - learning_rate:
5.0000e-04
Epoch 34/100
6/6             0s 37ms/step - loss:
0.0122 - mae: 0.0741 - val_loss: 0.0123 - val_mae: 0.0849 - learning_rate:
5.0000e-04
Epoch 35/100
6/6             0s 27ms/step - loss:
0.0118 - mae: 0.0718 - val_loss: 0.0124 - val_mae: 0.0870 - learning_rate:
5.0000e-04
Epoch 36/100
6/6             0s 28ms/step - loss:
0.0115 - mae: 0.0720 - val_loss: 0.0123 - val_mae: 0.0853 - learning_rate:
5.0000e-04
Epoch 37/100
6/6             0s 30ms/step - loss:
0.0122 - mae: 0.0734 - val_loss: 0.0122 - val_mae: 0.0839 - learning_rate:
5.0000e-04
Epoch 38/100
6/6             0s 25ms/step - loss:
0.0118 - mae: 0.0724 - val_loss: 0.0122 - val_mae: 0.0854 - learning_rate:
2.5000e-04
Epoch 39/100
6/6             0s 26ms/step - loss:
0.0117 - mae: 0.0729 - val_loss: 0.0122 - val_mae: 0.0864 - learning_rate:
2.5000e-04
Epoch 40/100
6/6             0s 30ms/step - loss:
0.0114 - mae: 0.0709 - val_loss: 0.0122 - val_mae: 0.0849 - learning_rate:
2.5000e-04
```

```
Epoch 41/100
6/6             0s 34ms/step - loss:
0.0113 - mae: 0.0697 - val_loss: 0.0122 - val_mae: 0.0850 - learning_rate:
2.5000e-04
Epoch 42/100
6/6             0s 29ms/step - loss:
0.0121 - mae: 0.0741 - val_loss: 0.0122 - val_mae: 0.0861 - learning_rate:
2.5000e-04
Epoch 43/100
6/6             0s 27ms/step - loss:
0.0115 - mae: 0.0709 - val_loss: 0.0123 - val_mae: 0.0865 - learning_rate:
2.5000e-04
Epoch 44/100
6/6             0s 26ms/step - loss:
0.0118 - mae: 0.0730 - val_loss: 0.0122 - val_mae: 0.0857 - learning_rate:
2.5000e-04
Epoch 45/100
6/6             0s 39ms/step - loss:
0.0110 - mae: 0.0706 - val_loss: 0.0123 - val_mae: 0.0869 - learning_rate:
2.5000e-04
Epoch 46/100
6/6             0s 27ms/step - loss:
0.0122 - mae: 0.0744 - val_loss: 0.0123 - val_mae: 0.0863 - learning_rate:
2.5000e-04
Epoch 47/100
6/6             0s 29ms/step - loss:
0.0119 - mae: 0.0731 - val_loss: 0.0122 - val_mae: 0.0861 - learning_rate:
2.5000e-04
Epoch 48/100
6/6             0s 31ms/step - loss:
0.0114 - mae: 0.0707 - val_loss: 0.0122 - val_mae: 0.0851 - learning_rate:
2.5000e-04
Epoch 49/100
6/6             0s 27ms/step - loss:
0.0115 - mae: 0.0705 - val_loss: 0.0122 - val_mae: 0.0857 - learning_rate:
1.2500e-04
Epoch 50/100
6/6             0s 29ms/step - loss:
0.0115 - mae: 0.0723 - val_loss: 0.0122 - val_mae: 0.0866 - learning_rate:
1.2500e-04
Epoch 51/100
6/6             0s 29ms/step - loss:
0.0116 - mae: 0.0722 - val_loss: 0.0122 - val_mae: 0.0866 - learning_rate:
1.2500e-04
Epoch 52/100
6/6             0s 29ms/step - loss:
0.0118 - mae: 0.0728 - val_loss: 0.0122 - val_mae: 0.0855 - learning_rate:
1.2500e-04
```

```
Epoch 53/100
6/6             0s 31ms/step - loss:
0.0112 - mae: 0.0684 - val_loss: 0.0122 - val_mae: 0.0858 - learning_rate:
1.2500e-04
7/7             1s 70ms/step
2/2             0s 32ms/step

Advanced LSTM Performance:
Training RMSE: 57.5060
Test RMSE: 67.1546
Training MAE: 39.5678
Test MAE: 50.1635
```

7 Results Visualization

This function `plot_training_history()` helps **visualize how each model learns** and compare their final performance side by side. It's an essential step to **diagnose overfitting, convergence, and model stability**.

- **Top-left (Loss plot):** Shows how training and validation losses evolve over epochs for each model. A small gap means good generalization; a widening gap signals overfitting.
- **Top-right (MAE plot):** Tracks Mean Absolute Error for both training and validation sets — useful to interpret model accuracy in real terms.
- **Bottom-left (RMSE bar chart):** Compares test RMSE across models — lower is better.
- **Bottom-right (MAE bar chart):** Compares test MAE for another view of model precision.
- Using **subplots (2×2)** offers a clean overview of learning behavior and final metrics in one figure.
- **Validation curves** (dashed lines) show how well models perform on unseen data during training.
- **Bar charts** provide quick comparison — ideal for presentations.

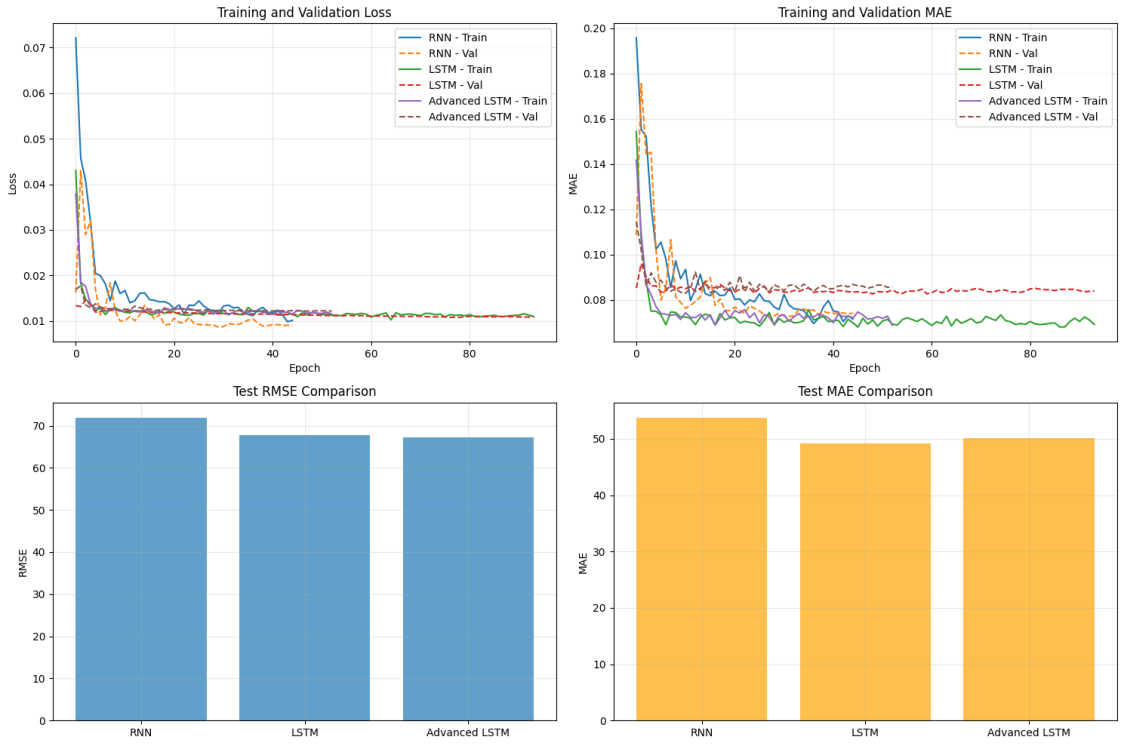
7.0.1 Alternatives & Tips

- Try **log scale** for loss if values vary widely.
- Add **smoothing** (e.g., moving average) to make noisy curves more readable.
- Save figures automatically with `plt.savefig()` for reporting.
- Consider using **TensorBoard** for more interactive monitoring in real time.

[33]:
```python
def plot_training_history(histories, model_names):
    """Plot training history for all models"""
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))

    # Plot training and validation loss
    for i, (history, name) in enumerate(zip(histories, model_names)):
```

```
        axes[0, 0].plot(history.history['loss'], label=f'{name} - Train')
        axes[0, 0].plot(history.history['val_loss'], label=f'{name} - Val',␣
↪linestyle='--')

    axes[0, 0].set_title('Training and Validation Loss')
    axes[0, 0].set_xlabel('Epoch')
    axes[0, 0].set_ylabel('Loss')
    axes[0, 0].legend()
    axes[0, 0].grid(True, alpha=0.3)

    # Plot MAE
    for i, (history, name) in enumerate(zip(histories, model_names)):
        axes[0, 1].plot(history.history['mae'], label=f'{name} - Train')
        axes[0, 1].plot(history.history['val_mae'], label=f'{name} - Val',␣
↪linestyle='--')

    axes[0, 1].set_title('Training and Validation MAE')
    axes[0, 1].set_xlabel('Epoch')
    axes[0, 1].set_ylabel('MAE')
    axes[0, 1].legend()
    axes[0, 1].grid(True, alpha=0.3)

    # Model comparison - Test RMSE
    model_names_list = list(models_results.keys())
    test_rmse_values = [models_results[name]['test_rmse'] for name in␣
↪model_names_list]

    axes[1, 0].bar(model_names_list, test_rmse_values, alpha=0.7)
    axes[1, 0].set_title('Test RMSE Comparison')
    axes[1, 0].set_ylabel('RMSE')
    axes[1, 0].grid(True, alpha=0.3)

    # Model comparison - Test MAE
    test_mae_values = [models_results[name]['test_mae'] for name in␣
↪model_names_list]

    axes[1, 1].bar(model_names_list, test_mae_values, alpha=0.7, color='orange')
    axes[1, 1].set_title('Test MAE Comparison')
    axes[1, 1].set_ylabel('MAE')
    axes[1, 1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

# Plot training histories
plot_training_history(
    [rnn_history, lstm_history, advanced_lstm_history],
```

```
    ['RNN', 'LSTM', 'Advanced LSTM']
)
```



7.1 Comparing Actual vs Predicted Values

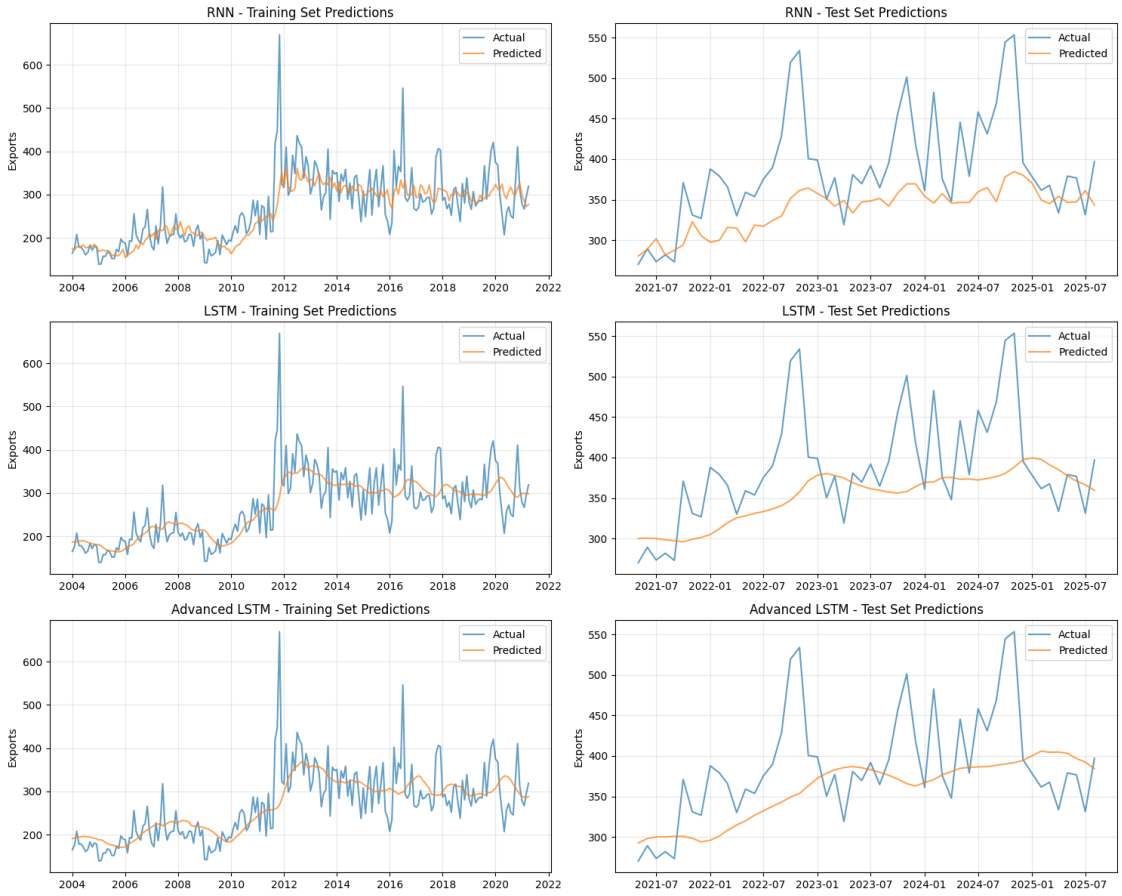
This function `plot_predictions()` visually compares **what the models predicted** versus **what actually happened** — both during training and testing phases. It's the most intuitive way to assess **forecast accuracy** and detect where models perform well or struggle.

- Creates a **grid of plots** — one row per model.
  - **Left column:** Model's fit on the *training data* (how well it learned past patterns).
  - **Right column:** Model's predictions on *test data* (how well it generalizes to unseen periods).
- Each plot overlays **actual values** (blue line) and **predicted values** (orange line) over time.
- The timeline (`train_dates` and `test_dates`) ensures alignment between real data and model outputs.
- Splitting into **training vs test visuals** helps spot **overfitting** — if the model fits training data perfectly but diverges in testing, it's memorizing rather than learning.
- Using **subplots per model** (RNN, LSTM, Advanced LSTM) makes performance comparison straightforward.
- **Transparency (`alpha=0.7`)** improves readability when lines overlap.

7.1.1 Tips & Tricks

- Add **forecast residuals** below each plot to highlight bias or lag.
- Use **scatter plots** (`actual vs predicted`) to check linearity of predictions.
- For presentations, focus on the **test plots** — they reflect real-world performance.
- Use `plt.savefig()` to export charts for reports.

[35]:
```
def plot_predictions(models_results, target_variable):
    """Plot predictions vs actual values for all models"""
    fig, axes = plt.subplots(len(models_results), 2, figsize=(15,␣
 ↪4*len(models_results)))

    # Create time indices for plotting
    train_dates = df.index[SEQUENCE_LENGTH:
 ↪SEQUENCE_LENGTH+len(models_results['RNN']['train_actual'])]
    test_dates = df.
 ↪index[SEQUENCE_LENGTH+len(models_results['RNN']['train_actual']):]

    for i, (model_name, results) in enumerate(models_results.items()):
        # Training predictions
        axes[i, 0].plot(train_dates, results['train_actual'], label='Actual',␣
 ↪alpha=0.7)
        axes[i, 0].plot(train_dates, results['train_pred'], label='Predicted',␣
 ↪alpha=0.7)
        axes[i, 0].set_title(f'{model_name} - Training Set Predictions')
        axes[i, 0].set_ylabel(target_variable)
        axes[i, 0].legend()
        axes[i, 0].grid(True, alpha=0.3)

        # Test predictions
        axes[i, 1].plot(test_dates, results['test_actual'], label='Actual',␣
 ↪alpha=0.7)
        axes[i, 1].plot(test_dates, results['test_pred'], label='Predicted',␣
 ↪alpha=0.7)
        axes[i, 1].set_title(f'{model_name} - Test Set Predictions')
        axes[i, 1].set_ylabel(target_variable)
        axes[i, 1].legend()
        axes[i, 1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

# Plot predictions for all models
plot_predictions(models_results, target_variable)
```



7.2 Analyzing Model Residuals

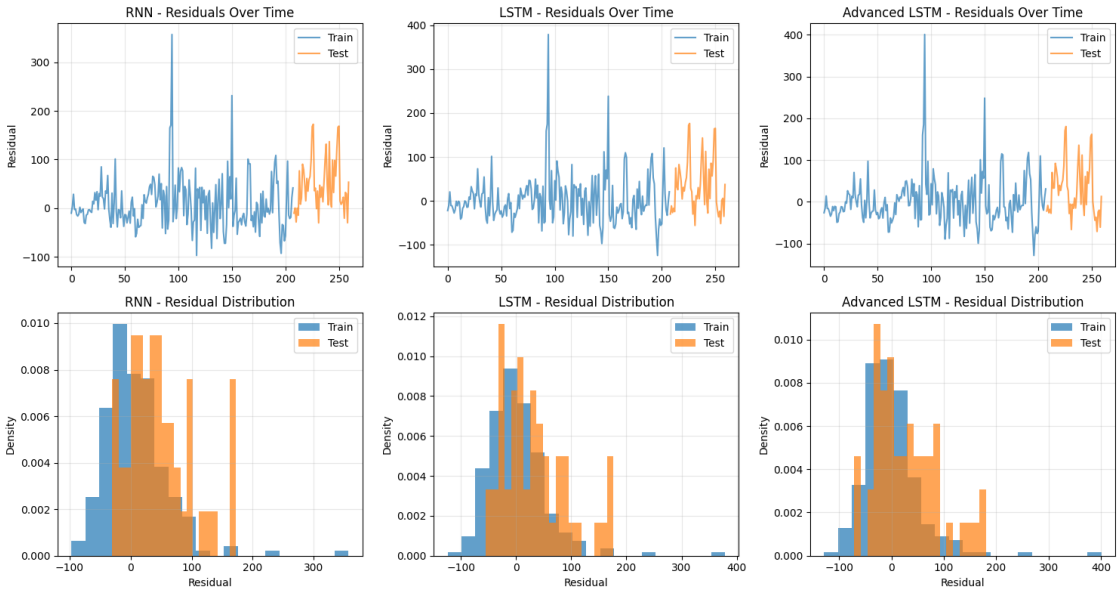
This function `plot_residuals()` helps us **understand the errors** our models make — not just how large they are, but also **how they behave** over time. Residuals (the difference between actual and predicted values) reveal where models consistently underperform or overfit.

- Computes **residuals** for each model: `Residual = Actual - Predicted`
- Creates **two rows of plots per model**:
  - **Top row:** Residuals over time — showing when the model tends to over- or under-predict.
  - **Bottom row:** Histogram of residuals — showing the *distribution* of errors (ideally centered around zero).
- Distinguishes between **training** and **testing** errors using color and labels.
- **Time-based residual plots** help detect *structural breaks* or *seasonal biases*.
- **Histogram plots** show if errors are symmetrically distributed — a key sign of an unbiased model.
- Using both **train** and **test residuals** side by side exposes potential **overfitting**.

7.2.1 Tips & Tricks

- Add a **rolling mean of residuals** to smooth short-term noise.
- Use **QQ-plots** (quantile-quantile) for checking normality of residuals.
- Large, consistent positive or negative residuals → model missing systematic patterns.
- Residuals with changing variance → possible **heteroscedasticity**, suggesting model refinement.

[37]:
```
def plot_residuals(models_results):
    """Plot residuals for all models"""
    fig, axes = plt.subplots(2, len(models_results),␣
 ↪figsize=(5*len(models_results), 8))

    for i, (model_name, results) in enumerate(models_results.items()):
        # Calculate residuals
        train_residuals = results['train_actual'].flatten() -␣
 ↪results['train_pred'].flatten()
        test_residuals = results['test_actual'].flatten() -␣
 ↪results['test_pred'].flatten()

        # Plot residuals over time
        axes[0, i].plot(train_residuals, alpha=0.7, label='Train')
        axes[0, i].plot(range(len(train_residuals), len(train_residuals) +␣
 ↪len(test_residuals)),
                        test_residuals, alpha=0.7, label='Test')
        axes[0, i].set_title(f'{model_name} - Residuals Over Time')
        axes[0, i].set_ylabel('Residual')
        axes[0, i].legend()
        axes[0, i].grid(True, alpha=0.3)

        # Plot residual distribution
        axes[1, i].hist(train_residuals, bins=20, alpha=0.7, label='Train',␣
 ↪density=True)
        axes[1, i].hist(test_residuals, bins=20, alpha=0.7, label='Test',␣
 ↪density=True)
        axes[1, i].set_title(f'{model_name} - Residual Distribution')
        axes[1, i].set_xlabel('Residual')
        axes[1, i].set_ylabel('Density')
        axes[1, i].legend()
        axes[1, i].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

# Plot residuals
plot_residuals(models_results)
```



8 Analyzing Model Performance

This function `analyze_model_performance()` compares how well different deep learning models (RNN, LSTM, and Advanced LSTM) performed on the same forecasting task. It gives both **numerical insights** (via metrics) and **interpretative clues** (like overfitting and model complexity).

8.1 What the Code Does

- Builds a **DataFrame** summarizing each model's key metrics:
  - **RMSE (Root Mean Squared Error):** Penalizes large errors, showing accuracy in magnitude.
  - **MAE (Mean Absolute Error):** Measures average prediction error in simpler terms.
- Calculates an **overfitting indicator**: Overfitting = ((Test RMSE - Train RMSE)/Train RMSE) * 100. A large gap signals the model fits training data too tightly.
- Identifies the **best-performing model** (lowest Test RMSE).
- Prints **model complexity** — total number of trainable parameters — to help balance accuracy vs efficiency.

8.2 Why These Choices

- RMSE and MAE together give a **complementary view** of accuracy.
- The overfitting measure is simple yet effective for model comparison.
- Counting parameters helps evaluate **model scalability** and **computational cost** — important for deployment in production environments like central banks.

8.3 Alternatives & Tips

- Could include **R^2 (coefficient of determination)** for interpretability.
- Use **cross-validation** for more reliable generalization testing.
- For model selection, techniques like **Akaike Information Criterion (AIC)** or **Bayesian Optimization** can be explored.
- Always look at both **performance and complexity** — a slightly less accurate but much simpler model may be preferred in practice.

[39]:
```
def analyze_model_performance():
    """Analyze and compare model performance"""
    print("="*70)
    print("MODEL PERFORMANCE SUMMARY")
    print("="*70)

    # Create performance comparison table
    performance_df = pd.DataFrame({
        'Model': list(models_results.keys()),
        'Train RMSE': [results['train_rmse'] for results in models_results.
 ↪values()],
        'Test RMSE': [results['test_rmse'] for results in models_results.
 ↪values()],
        'Train MAE': [results['train_mae'] for results in models_results.
 ↪values()],
        'Test MAE': [results['test_mae'] for results in models_results.values()]
    })

    # Calculate overfitting indicator
    performance_df['Overfitting (RMSE)'] = (performance_df['Test RMSE'] -␣
 ↪performance_df['Train RMSE']) / performance_df['Train RMSE'] * 100

    print(performance_df.round(4))

    # Best model analysis
    best_model_idx = performance_df['Test RMSE'].idxmin()
    best_model = performance_df.loc[best_model_idx, 'Model']
    best_rmse = performance_df.loc[best_model_idx, 'Test RMSE']

    print(f"\nBest performing model: {best_model}")
    print(f"Best Test RMSE: {best_rmse:.4f}")

    # Statistical significance test (simplified)
    print(f"\nModel Complexity Analysis:")
    print(f"RNN Parameters: {rnn_model.count_params():,}")
    print(f"LSTM Parameters: {lstm_model.count_params():,}")
    print(f"Advanced LSTM Parameters: {advanced_lstm_model.count_params():,}")

    return performance_df
```

```
performance_summary = analyze_model_performance()
```

```
======================================================================
MODEL PERFORMANCE SUMMARY
======================================================================
           Model  Train RMSE  Test RMSE  Train MAE  Test MAE  \
0            RNN     53.2121    71.9380    36.7714   53.7698
1           LSTM     55.1235    67.7839    37.9461   49.1420
2  Advanced LSTM     57.5060    67.1546    39.5678   50.1635

   Overfitting (RMSE)
0             35.1910
1             22.9675
2             16.7785

Best performing model: Advanced LSTM
Best Test RMSE: 67.1546

Model Complexity Analysis:
RNN Parameters: 8,951
LSTM Parameters: 31,901
Advanced LSTM Parameters: 54,651
```

9 Future Forecasting

This section demonstrates how to use the **trained deep learning model** (RNN, LSTM, or Advanced LSTM) to make **future forecasts** for the next 12 months — a practical step where model insights turn into actionable predictions.

- The function `forecast_future()` takes the **last observed sequence** of data and predicts future values one step at a time.
- Each new prediction is **fed back** into the sequence to forecast the next time step — a process known as **recursive forecasting**.
- Predictions are **inverse-transformed** to return them to their original scale.
- The code then:
  - Selects the **best-performing model** based on lowest test RMSE,
  - Forecasts the next 12 months,
  - Plots the **historical data vs. forecasted values**, and
  - Adds a **95% confidence interval** to visualize prediction uncertainty.

9.1 Why These Choices

- Recursive forecasting reflects how real-world forecasting works — predicting one period ahead at a time.
- Using **MinMaxScaler's inverse transformation** restores interpretability to the model's scaled outputs.
- A **12-month horizon** is a realistic choice for economic and financial time series.
- Confidence intervals (using $\pm 1.96 \times$ std of residuals) offer a quick uncertainty estimate.

9.2 Alternatives & Tips

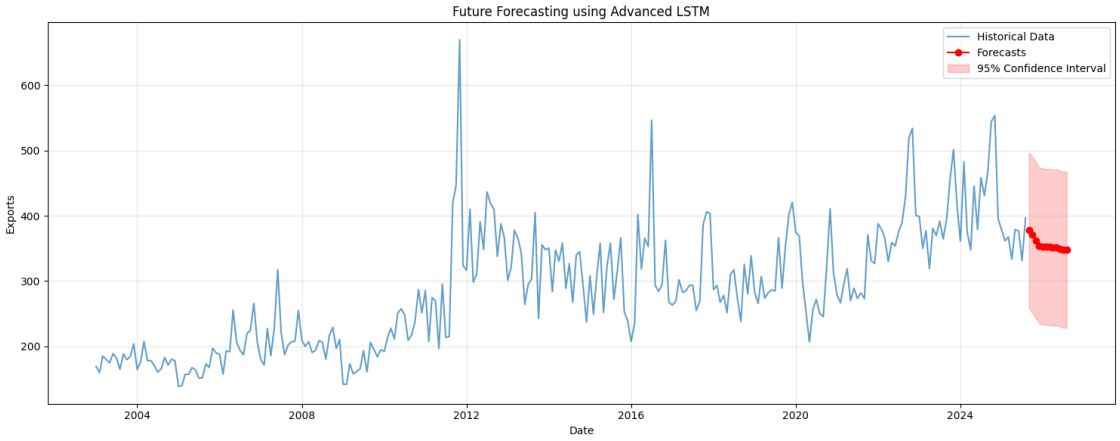
- For more robust forecasting, try **multi-step models** that predict several future points at once.
- Use **bootstrapping or Monte Carlo dropout** for more accurate confidence intervals.
- Extend forecasts dynamically using real-time data feeds.
- Always visually inspect forecast plots — patterns like **trend drift** or **oversmoothing** may signal retraining needs.

[41]:
```python
def forecast_future(model, last_sequence, n_periods=12, scaler=scaler):
    """
    Generate future forecasts using trained model

    Args:
        model: trained Keras model
        last_sequence: last sequence from training data
        n_periods: number of periods to forecast
        scaler: fitted scaler for inverse transformation

    Returns:
        array of forecasted values
    """
    forecasts = []
    current_sequence = last_sequence.copy()

    for _ in range(n_periods):
        # Predict next value
        next_pred = model.predict(current_sequence.reshape(1, SEQUENCE_LENGTH,␣
↪1))
        forecasts.append(next_pred[0, 0])

        # Update sequence (remove first element, add prediction)
        current_sequence = np.roll(current_sequence, -1)
        current_sequence[-1] = next_pred[0, 0]

    # Inverse transform forecasts
    forecasts = np.array(forecasts).reshape(-1, 1)
    forecasts_inv = scaler.inverse_transform(forecasts)
```

```
    return forecasts_inv.flatten()

# Generate future forecasts using the best model
best_model_name = performance_summary.loc[performance_summary['Test RMSE'].
 ↪idxmin(), 'Model']
print(f"Using {best_model_name} for future forecasting...")

if best_model_name == 'RNN':
    best_model = rnn_model
elif best_model_name == 'LSTM':
    best_model = lstm_model
else:
    best_model = advanced_lstm_model

# Get the last sequence from the scaled data
last_sequence = scaled_data[-SEQUENCE_LENGTH:]

# Forecast next 12 months
future_forecasts = forecast_future(best_model, last_sequence, n_periods=12)

# Create future dates
last_date = df.index[-1]
future_dates = pd.date_range(start=last_date + pd.DateOffset(months=1),␣
 ↪periods=12, freq='MS')

# Plot historical data and forecasts
plt.figure(figsize=(15, 6))

# Plot historical data
plt.plot(df.index, df[target_variable], label='Historical Data', alpha=0.7)

# Plot forecasts
plt.plot(future_dates, future_forecasts, label='Forecasts', color='red',␣
 ↪marker='o')

# Add confidence interval (simplified)
forecast_std = np.std(models_results[best_model_name]['test_actual'] -␣
 ↪models_results[best_model_name]['test_pred'])
plt.fill_between(future_dates,
                 future_forecasts - 1.96 * forecast_std,
                 future_forecasts + 1.96 * forecast_std,
                 alpha=0.2, color='red', label='95% Confidence Interval')

plt.title(f'Future Forecasting using {best_model_name}')
plt.xlabel('Date')
plt.ylabel(target_variable)
plt.legend()
```

```
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Print forecast values
print(f"\nFuture Forecasts for {target_variable}:")
print("-" * 40)
for date, forecast in zip(future_dates, future_forecasts):
    print(f"{date.strftime('%Y-%m')}: {forecast:.2f}")
```

```
Using Advanced LSTM for future forecasting…
1/1             0s 31ms/step
1/1             0s 45ms/step
1/1             0s 35ms/step
1/1             0s 32ms/step
1/1             0s 31ms/step
1/1             0s 41ms/step
1/1             0s 35ms/step
1/1             0s 45ms/step
1/1             0s 42ms/step
1/1             0s 31ms/step
1/1             0s 42ms/step
1/1             0s 47ms/step
```




```
Future Forecasts for Exports:
----------------------------------------
2025-09: 377.86
2025-10: 370.97
2025-11: 362.29
2025-12: 354.24
2026-01: 353.09
```

```
2026-02: 352.45
2026-03: 352.15
2026-04: 351.28
2026-05: 351.51
2026-06: 349.62
2026-07: 347.85
2026-08: 347.88
```

check

10 (Optional) Key Insights and Recommendations

This final section wraps up the notebook by summarizing everything achieved and presenting key insights from model training and evaluation. It provides a concise overview of the workflow, model comparison, and practical recommendations.

The code prints a structured summary — confirming data preparation, models trained, best-performing model, and visualization results — followed by a detailed breakdown of insights across six key areas:

- **Model performance:** Highlights the best and worst models with their RMSE gap.
- **Overfitting analysis:** Evaluates how well models generalize beyond training data.
- **Data characteristics:** Summarizes the dataset's length, range, and variability.
- **Model complexity:** Compares architectures by parameter count and computational load.
- **Business recommendations:** Suggests operational strategies for production forecasting.
- **Technical recommendations:** Proposes next steps such as tuning, multivariate modeling, and advanced architectures.

This section is valuable for communicating findings to both **technical** and **non-technical audiences**, transforming deep learning outcomes into actionable insights for decision-making.

[44]:
```python
# Final summary
print("="*70)
print("NOTEBOOK COMPLETION SUMMARY")
print("="*70)
print(f" Data loaded and processed: {len(df)} time points")
print(f" Models trained: 3 different architectures")
print(f" Best model: {best_model_name}")
print(f" Future forecasts: 12 months generated")
print(f" Visualization: Comprehensive plots created")
print(f" Advanced techniques: Introduced and demonstrated")

"""Generate key insights from the analysis"""
print("="*70)
print("KEY INSIGHTS AND RECOMMENDATIONS")
print("="*70)

# Performance insights
```

```
best_model = performance_summary.loc[performance_summary['Test RMSE'].idxmin(),␣
 ↪'Model']
worst_model = performance_summary.loc[performance_summary['Test RMSE'].
 ↪idxmax(), 'Model']

print(f"1. MODEL PERFORMANCE:")
print(f"   • Best model: {best_model}")
print(f"   • Worst model: {worst_model}")
print(f"   • Performance gap: {performance_summary['Test RMSE'].max() -␣
 ↪performance_summary['Test RMSE'].min():.4f} RMSE")

# Overfitting analysis
print(f"\n2. OVERFITTING ANALYSIS:")
for _, row in performance_summary.iterrows():
    overfitting = row['Overfitting (RMSE)']
    status = "HIGH" if overfitting > 10 else "MODERATE" if overfitting > 5 else␣
 ↪"LOW"
    print(f"   • {row['Model']}: {overfitting:.1f}% overfitting ({status})")

    # Data insights
    print(f"\n3. DATA CHARACTERISTICS:")
    print(f"   • Time series length: {len(df)} months")
    print(f"   • Sequence length used: {SEQUENCE_LENGTH} months")
    print(f"   • Target variable: {target_variable}")
    print(f"   • Data range: {df[target_variable].min():.2f} to␣
 ↪{df[target_variable].max():.2f}")
    print(f"   • Data volatility (std): {df[target_variable].std():.2f}")

    # Model complexity insights
    print(f"\n4. MODEL COMPLEXITY:")
    models_params = {
        'RNN': rnn_model.count_params(),
        'LSTM': lstm_model.count_params(),
        'Advanced LSTM': advanced_lstm_model.count_params()
    }

    for model_name, params in models_params.items():
        complexity = "HIGH" if params > 10000 else "MODERATE" if params > 5000␣
 ↪else "LOW"
        print(f"   • {model_name}: {params:,} parameters ({complexity}␣
 ↪complexity)")

    # Business recommendations
    print(f"\n5. BUSINESS RECOMMENDATIONS:")
    print(f"   • Use {best_model} for production forecasting")
    print(f"   • Implement ensemble methods combining multiple models")
```

```
    print(f"   • Consider external factors (seasonality, economic indicators)")
    print(f"   • Regular model retraining (monthly/quarterly)")
    print(f"   • Monitor forecast accuracy and drift over time")

    # Technical recommendations
    print(f"\n6. TECHNICAL RECOMMENDATIONS:")
    print(f"   • Experiment with different sequence lengths (6-24 months)")
    print(f"   • Try multivariate models including other trade variables")
    print(f"   • Implement hyperparameter tuning (grid search, Bayesian␣
↪optimization)")
    print(f"   • Consider advanced architectures (GRU, Transformer, CNN-LSTM)")
    print(f"   • Add feature engineering (moving averages, seasonality)")
```

```
======================================================================
NOTEBOOK COMPLETION SUMMARY
======================================================================
 Data loaded and processed: 272 time points
 Models trained: 3 different architectures
 Best model: Advanced LSTM
 Future forecasts: 12 months generated
 Visualization: Comprehensive plots created
 Advanced techniques: Introduced and demonstrated
======================================================================
KEY INSIGHTS AND RECOMMENDATIONS
======================================================================
1. MODEL PERFORMANCE:
   • Best model: Advanced LSTM
   • Worst model: RNN
   • Performance gap: 4.7833 RMSE

2. OVERFITTING ANALYSIS:
   • RNN: 35.2% overfitting (HIGH)

3. DATA CHARACTERISTICS:
   • Time series length: 272 months
   • Sequence length used: 12 months
   • Target variable: Exports
   • Data range: 138.80 to 669.40
   • Data volatility (std): 92.84

4. MODEL COMPLEXITY:
   • RNN: 8,951 parameters (MODERATE complexity)
   • LSTM: 31,901 parameters (HIGH complexity)
   • Advanced LSTM: 54,651 parameters (HIGH complexity)

5. BUSINESS RECOMMENDATIONS:
   • Use Advanced LSTM for production forecasting
```

- Implement ensemble methods combining multiple models
- Consider external factors (seasonality, economic indicators)
- Regular model retraining (monthly/quarterly)
- Monitor forecast accuracy and drift over time

6. TECHNICAL RECOMMENDATIONS:
   - Experiment with different sequence lengths (6-24 months)
   - Try multivariate models including other trade variables
   - Implement hyperparameter tuning (grid search, Bayesian optimization)
   - Consider advanced architectures (GRU, Transformer, CNN-LSTM)
   - Add feature engineering (moving averages, seasonality)
   - LSTM: 23.0% overfitting (HIGH)

3. DATA CHARACTERISTICS:
   - Time series length: 272 months
   - Sequence length used: 12 months
   - Target variable: Exports
   - Data range: 138.80 to 669.40
   - Data volatility (std): 92.84

4. MODEL COMPLEXITY:
   - RNN: 8,951 parameters (MODERATE complexity)
   - LSTM: 31,901 parameters (HIGH complexity)
   - Advanced LSTM: 54,651 parameters (HIGH complexity)

5. BUSINESS RECOMMENDATIONS:
   - Use Advanced LSTM for production forecasting
   - Implement ensemble methods combining multiple models
   - Consider external factors (seasonality, economic indicators)
   - Regular model retraining (monthly/quarterly)
   - Monitor forecast accuracy and drift over time

6. TECHNICAL RECOMMENDATIONS:
   - Experiment with different sequence lengths (6-24 months)
   - Try multivariate models including other trade variables
   - Implement hyperparameter tuning (grid search, Bayesian optimization)
   - Consider advanced architectures (GRU, Transformer, CNN-LSTM)
   - Add feature engineering (moving averages, seasonality)
   - Advanced LSTM: 16.8% overfitting (HIGH)

3. DATA CHARACTERISTICS:
   - Time series length: 272 months
   - Sequence length used: 12 months
   - Target variable: Exports
   - Data range: 138.80 to 669.40
   - Data volatility (std): 92.84

4. MODEL COMPLEXITY:

- RNN: 8,951 parameters (MODERATE complexity)
- LSTM: 31,901 parameters (HIGH complexity)
- Advanced LSTM: 54,651 parameters (HIGH complexity)

5. BUSINESS RECOMMENDATIONS:
   - Use Advanced LSTM for production forecasting
   - Implement ensemble methods combining multiple models
   - Consider external factors (seasonality, economic indicators)
   - Regular model retraining (monthly/quarterly)
   - Monitor forecast accuracy and drift over time

6. TECHNICAL RECOMMENDATIONS:
   - Experiment with different sequence lengths (6-24 months)
   - Try multivariate models including other trade variables
   - Implement hyperparameter tuning (grid search, Bayesian optimization)
   - Consider advanced architectures (GRU, Transformer, CNN-LSTM)
   - Add feature engineering (moving averages, seasonality)